6809 Forth — Open Items Checklist (Part 2 of 2: Unit Test Framework)

This is part 2 of 2, picking up exactly where part 1 leaves off: the assembly-level unit test framework's own introduction, and every glossary-section test group (3.1 through 3.13 so far) built on it since, including every bug found and fixed via real MAME testing along the way. See part 1 for the initial 6809 port and its core bug fixes that preceded this work.

Known bugs — resolved since the original checklist (continued)

- **New capability, not a bug fix: an assembly-level unit test framework added, with its first test (`TSTDUP`) written.** Gated by a new conditional-assembly flag, `UNITTESTS` , matching this file's established `IFEQ` convention (`0` = included, `1` = excluded entirely). Two insertion points: a `JSR TSTRUNNER` call at the true end of `COLDSTRT` (after `INITSERIAL` , before `JMP COLD` - at that point `U` / `S` are already valid, since `COLDSTRT` sets them at its very start, but nothing else - `APPVARS` / `DPHERE` / `CODEHERE` / `LATEST` / `BASE` - has been initialized yet), and the test framework's own body, living in what was previously unused ROM space right after `INOUT` 's shadow (`USROMSTRT` , `$C100`) - pure `FILL` padding before this existed. `UNITTESTS=1` reverts this block to exactly that padding, computed automatically via the `ROM` label rather than a fixed byte count (`FILL $FF,BASEDICT-ROM` , was `BASEDICT- USROMSTRT`), so it stays correct either way without needing hand-adjustment when the test code's size changes. Each test is independent by construction, per the stated design principle: it saves the data stack pointer (`U`) before touching anything and unconditionally restores it at the end regardless of pass or fail, so a failed assertion mid-test can never leave stack residue for the next test to inherit. Test scratch variables live at the very start of `APPVARS` - safe only because tests run strictly before `COLD` initializes `VARHERE` to that same address, which correctly and immediately re-purposes that space afterward; this is a real constraint worth remembering if the call site ever moves. Reporting is shared across every test via `TSTREPORT` rather than duplicated per test: prints the test's name (a counted string, via `COUNT` + `TYPE` - the same mechanism `BADWORD` itself uses to print a failing word), then " OK" or " FAIL", then a `CR` . `TSTDUP` verifies both the stack's contents after `DUP` (the duplicate and the original both equal a deliberately non- trivial pushed test value - not `0` , `1` , or `-1` , so a test that only appears to pass due to a special-cased value would be caught - and a sentinel pushed beneath is confirmed undisturbed) and the data stack pointer's movement (exactly one cell, 2 bytes - `DUP` 's own net effect, isolated from the

two setup pushes by capturing `U` immediately before and after the `JSR DUP` specifically, not around the whole test). `TSTRUNNER` and `TSTSTACK` are structured as extensible dispatchers (`JSR` a list of test groups, `JSR` a list of tests within each group) so more tests and more groups can be added without restructuring what's here. Verified across the full 2x2 matrix of `SERIALPOLL` / `UNITTESTS` combinations: zero duplicate symbols, dictionary chain intact (224 entries) in all four; byte-exact split-file reassembly. **MAME-CONFIRMED**: activated by the user, `TSTDUP` runs successfully and reports the correct message to the terminal - both the framework itself and `DUP` 's own test are now verified on real hardware, not just structurally. Whatever earlier concern prompted excluding this block was resolved by other, unrelated fixes made since it was first written.

- **Three real bugs found by inspection (not MAME) and fixed: `DEFER` , `2CONSTANT` , and `MARKER` all had the identical self-referential PFA bug already confirmed and fixed in `CONSTANT` via the MAME debugger.** Flagged by a parallel 68000 port of this codebase, which spotted the same construction pattern (`LDD CODEHERE / PSHU D / JSR CODECOMMA` , writing the PFA field's own current address into itself before the real value is appended two bytes later) recurring in these three defining words, unfixed. Verified each by inspection rather than deferred to hardware testing, since this is a deterministic compile-time address computation, not a timing/register- interaction issue - the same reasoning already used to confirm `VALUEW` was *not* affected when `CONSTANT` was first fixed. Traced both the compile-time construction and the runtime consumer for each: `DODEFER` 's `LDD ,X` would have jumped to the PFA's own address on execution of any `DEFER` 'd word (crash); `DOMARKER` 's four fixed-offset reads (`,X` / `2,X` / `4,X` / `6,X`) would have restored garbage `DPHERE` / `CODEHERE` / `VARHERE` / `LATEST` values - the most severe of the three, since using a `MARKER` -created word would corrupt live dictionary state, not just misbehave locally. Also checked, and confirmed genuinely unaffected rather than assumed safe by association: `DEFER@` / `DEFER!` (go through `TOBODY` , a runtime offset computation from an existing xt - structurally immune, never compiles a self- referential field), `IS` / `ACTION-OF` (construct no PFA at all, just locate a word and call `DEFERSTORE` / `DEFERFETCH`), and `BUFFER:` (`BUFFERCOLON` , checked because it sits directly next to `2CONSTANT` in the source - uses the `VALUEW` -style pattern, PFA into `VARHERE` rather than `CODEHERE` , and `VALLOT` only advances `VARHERE` without ever writing through the PFA, so no self-reference exists there either). Fixed all three with the same `ADDD #2` pattern already established for `CONSTANT` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME - the mechanism is identical to the already- hardware-confirmed `CONSTANT` bug, but these three fixes themselves are still unverified against real execution.

- **Real bug found by inspection (not MAME) and fixed: `2@` (`DFETCH`) read the two cells in the opposite order `2!` (`DSTORE`) actually writes them, so a `2! 2@` round trip swapped the values.** Flagged by the parallel 68000 port. Traced both routines precisely: `DSTORE` writes `x1` to the low address and `x2` to the high address; `DFETCH` was reading the high address first (pushing it deep) and the low address second (pushing it on top) - the reverse order. Confirmed via a concrete round-trip trace (`[x1, x2, a-addr]` before `2!` came back as `[x2, x1]` after `2@`), which is deterministic and fully verifiable from source - no MAME needed to establish it, same reasoning as the `MARKER` / `2CONSTANT` / `DEFER` PFA bugs. `2!` itself was already correct and internally consistent; only `2@` 's read order was wrong. Fixed by swapping `DFETCH` 's two `LDD` / `PSHU` pairs to read low-then-high instead of high-then-low. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Simplification (not a bug fix): the `PULS X` / `PSHS X` pair left over in `LEAVE` from the earlier `DOTEST` -class fix was a genuine no-op and has been removed.** Flagged by the parallel 68000 port, which dropped it rather than port a verified no-op forward. Confirmed by inspection: nothing runs between the pop and the push (no call, no other touch of `S` or `X`), so `X` held exactly what it held before the pop and `PSHS X` restored `S` to exactly where it already was - `RTS` would find the same return address there either way. Unlike `DOTEST` 's own deferred `PULS X` (genuinely load-bearing - its value feeds `LDD ,X` / `LEAX D,X` afterward), `LEAVE` never uses `X` 's value for anything; the pair was vestige of the pre-fix structure (where `PULS X` ran first and `PSHS X` was needed to restore `S` afterward), not something the fix itself required. `LEAVE` is now just `LDD #TRUEV / STD 6,S / RTS` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Simplification (not a bug fix): `U.` (`UDOT`) and `U.R` (`UDOTR`) each had a literal pop-then-immediate-push-back on the value being formatted - a genuine no-op, removed.** Flagged by the parallel 68000 port, same class as the `LEAVE` finding immediately above. Confirmed by inspection: in both routines, `PULU D` immediately followed by `PSHU D` with nothing in between leaves `U` exactly as it was: the following `LDD #0 / PSHU D` (building the double-number `NUMGT` needs) pushes `0` on top of the value either way, whether or not it was ever popped and pushed back first. In `UDOTR` specifically, only the *second* pop/push pair (the value) was the no-op - the first (`PULU D / STD DRWIDTH` , consuming the width argument) is genuinely functional and was left untouched. `UDOT` now starts directly with `LDD #0` ; `UDOTR` now goes straight from the width pop to `LDD #0` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file

reassembly.

- **Real bug found by inspection (not MAME) and fixed: `UNESCAPEW` double-decremented `UESRCLEN` for a lone trailing backslash, underflowing the counter so the loop would run past the end of the intended input.** Flagged by the parallel 68000 port. Traced precisely: on encountering `\`, the escape-handling path decrements `UESRCLEN` once (correctly, accounting for the backslash itself) and checks `BEQ` - if the backslash was the *last* character in the input, `UESRCLEN` is now correctly `0`, but the branch fell into the shared `UEPLAIN` (plain-character) path, which decrements `UESRCLEN` a second time for a character that doesn't exist. `0 - 1` underflows to `$FFFF` (nonzero), so the next `UELOOP` pass's `BEQ UEDONE` never fires and the loop reads memory past the intended input. Confirmed the normal paths are unaffected: an ordinary plain character decrements exactly once (`BNE UEPLAIN` from the top); a real two-character escape like `\n` decrements once for the backslash and once more via the shared `UEEMIT` path for the second character - correctly 2 total for 2 characters consumed. The bug is specific to the lone-trailing-backslash case, where the escape branch's own decrement already accounted for the backslash before falling into a path that decrements again. Fixed by giving that one case its own landing point, `UELASTBS` - emits the backslash literally (`A` still holds it from the earlier `LDA ,X+`, so nothing needs reloading) without re-decrementing `UESRCLEN`, since it's already correctly `0`. `UEPLAIN` itself, still used by the normal plain-character path, was left completely untouched. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Design change (not a bug fix): `UNLOOP` is now a true no-op, resolving the `UNLOOP` / `EXIT` return-stack corruption from the prior turn.** Reasoning: traced `EXIT` (via `EXITUNLOOP`) across every scenario - no enclosing `DO` (compiled count `0`, the unwind loop never executes, falls straight through to a plain return), one enclosing `DO` with no prior manual `UNLOOP` (discards the correct, still-intact 8-byte frame). Both correct. The only broken combination was `UNLOOP` immediately before `EXIT`: `UNLOOP`'s own discard (6 of 8 bytes) left the frame partially gone, then `EXITUNLOOP` unconditionally discarded a full 8 more, overshooting into whatever sat beneath - typically the enclosing word's own caller's return address - branching into random memory on return, requiring a soft reboot to recover. Given `UNLOOP` has no legitimate use other than immediately preceding an exit from the definition, and `EXIT` already handles that correctly and automatically on its own, the conflict is resolved by removing the second, redundant discard mechanism rather than by making `EXIT`'s runtime detect how much of the frame remains (a more complex, riskier change). Checked for a new failure mode before applying: `UNLOOP` called without an immediate exit, then falling

through to `LOOP` / `+LOOP` again - already broken before this change too (`DOTEST` / `DOPLUSTEST` already expect the frame to still be there), so this introduces no new risk, and actually removes one instance of it. `UNLOOP` is now just `RTS` . Documentation updated to match: `UNLOOP` 's glossary entry rewritten to describe the no-op and why, kept in the dictionary for source compatibility with code written for other Forth systems where `EXIT` doesn't auto-unwind; `EXIT` 's own entry given a cross-reference noting `UNLOOP` isn't required beforehand. The Assembler Source appendix (Section 8.13, Control Flow) was updated to match the real source rather than left stale. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly; both new documentation passages confirmed present in the rendered PDF. Not yet confirmed via MAME.

- **Real bug found by inspection (not MAME) and fixed: `CASE` pushed a `0` onto the compile-time `U` stack that nothing downstream ever consumed, causing every `CASE...ENDCASE` construct - even the simplest, with no `OF` clauses at all - to throw `-22` at the closing `;` .** Reported via `: Nname CASE 0 OF ENDOF ENDCASE ;` ; traced with the debugger to a `CSP` mismatch of exactly one stray cell. Verified the full compile- time lifecycle: `CASE` pushes `[0, TAGCASE]` ; `OF` pushes `[placeholder-addr, TAGOF]` ; `ENDOF` pops exactly those two and pushes its own `[NEWFLD, TAGENDOF]` ; `ENDCASE` loops popping `TAGENDOF` / `NEWFLD` pairs until it sees `TAGCASE` , then stops. None of `OF` / `ENDOF` / `ENDCASE` ever read or pop past `TAGCASE` - the `0 CASE` pushed beneath it is structurally unreachable by every consumer, leaving it permanently stranded one cell below where `CSP` (set by `:` before `CASE` ever runs) expects the depth to return to. `;` compares against `CSP` and correctly detects the mismatch every time. This implementation's `ENDCASE` uses a `TAGCASE` -scan approach entirely, with no counter involved anywhere in its actual logic - the stray `0` looks like a genuine leftover from an earlier, counter-based design that was never fully removed when the scan-based approach replaced it. Fixed by removing `CASEW` 's `LDD #0 / PSHU D` - now just `LDD #TAGCASE / PSHU D / RTS` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Correction to an earlier fix, found via a real MAME test (`MULT-TABLE`) and confirmed by inspection: `JWORD` 's offset, "fixed" to `10,S` a few turns ago, was itself wrong.** `J` returned a fixed value (the outer loop's limit) on every iteration instead of the progressing index - `11 1 DO 11 1 DO I J * . LOOP CR LOOP` printed the same row ten times instead of a real multiplication table. The earlier fix incorrectly assumed `DOSETUP` 's own `JSR` -pushed return address persists on `S` after each nesting level - it doesn't, since `DOSETUP` 's own `RTS` pops and consumes it to jump into the loop body, so

it never actually occupies a slot for `J` to count past. Re-traced by counting every real push and pop across a nested `DO` rather than assuming a fixed frame size per level: after the inner `DOSETUP` finishes, `S` is `[inner-index@0][inner-limit@2] [inner-leave@4][outer-index@6][outer-limit@8][outer-leave@10]`; `J`'s own `JSR` pushes one more cell on top, landing outer-index at `8`. Offset `10` (the earlier fix) lands on outer-limit instead - a value that never changes across outer iterations, exactly matching the observed symptom. `IWORD`'s own offset (`2`) was re-checked against this same corrected reasoning and confirmed still correct - it only ever sits one frame deep, not two, so the earlier error (specific to counting across a second, nested `DOSETUP` call) doesn't apply there. `JWORD` is now `LDD 8,S`. This is being logged transparently as a correction to a prior fix, not presented as if it were right the first time - worth being honest that even a change I was confident in and verified structurally at the time turned out to need real hardware testing to actually confirm. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found via a real MAME test (`CREATE DOES>`) and fixed: `CREATE` had the same self-referential PFA-pointer bug already confirmed and fixed in `CONSTANT`, `DEFER`, `2CONSTANT`, and `MARKER` - but was never on the previously-flagged list, so it went unfixed until now.** Reported via `: ENUM CREATE , DOES> @ ;` - `DOES>` returned the address one cell before the value `,` actually stored, instead of the value's own address. Traced precisely: `CREATE` compiles `[JSR DODOES][FDB DOESRT0] [FDB <CODEHERE's current value>]` - the same `LDD CODEHERE` - without- `+2` pattern as the other four, so the PFA-pointer field pointed at itself instead of two bytes further on, where `,` appends the real value next. Confirmed the runtime consequence through `DODOES` itself: it reads the *value stored at* the PFA-pointer field and pushes that (by design - this is how `CONSTANT`'s indirection works correctly, `DODOES` following a pointer to the real data rather than holding the data directly). With the field self-referencing, `DODOES` pushed the field's own address instead of following through to where the real value lives - exactly "the address before the 16-bit constant" instead of "the address 1 cell further on," matching the report precisely. Checked `VARIABLE` (the structurally similar routine immediately below `CREATE`, sharing the same `DOESRT0` trampoline setup) for the same bug rather than assume safety from resemblance alone - confirmed genuinely unaffected, since it uses the `VALUEW`-style pattern (`VARHERE`, a separate region) rather than `CODEHERE` self-reference. Also confirmed `SETDOES` needs no change - it patches a different field entirely (the behavior field, 2 bytes past `JSR DODOES`), fully independent of the PFA-pointer field this bug affects. Fixed with the same `ADDD #2` pattern already established for the other four. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **False alarm, resolved: a reported `2CONSTANT` / `2@` discrepancy (`666 777 2CONSTANT cc0` returning `666 $700C` instead of `666 777` ; a separate `2@` test on a `2VARIABLE` appearing to read back in reverse order) turned out to be testing against a stale binary, not a live bug.** Traced the reported symptom carefully against the current source before this was resolved: `CREATE` 's `+2` fix (this session, correcting the same self-referential PFA-pointer class as `CONSTANT` / `DEFER` / `2CONSTANT` / `MARKER`) and `DFETCH` 's low-then-high read order (fixed to match `DSTORE` several turns earlier, flagged by the parallel 68000 port) both checked out correctly on paper against the reported symptom - neither reproduced the described behavior in static trace, which was the first signal something didn't add up. Confirmed only one `DFETCH` definition exists in the file (no stale duplicate). Resolved once the user rebuilt against the current `forth6809.asm` /split files and re-tested: the updated binary shows neither problem. Both are confirmed fixed by the prior `CREATE` and `DFETCH` changes already logged above - no additional code change was needed. Recorded here explicitly so the history is clear: this was a real, reasonable bug report at the time, not a mistake in reporting it - the fix had simply already landed in a turn the tested binary predated.
- **Real, significant bug found via MAME debugger and fixed: `QLOOP` unconditionally reset `STATE` to interpret mode at the start of every line, silently breaking any colon definition split across more than one line of input.** Reported via `: test0 TRUE IF ." Hy" ELSE ." Hee" THEN ;` compiling correctly on one line but failing with a `-22` (CSP mismatch) when split across several. `COLON` sets `STATE=-1` and `SEMI` sets it back to `0` (after checking `CSP`) - the correct, sole places `STATE` should change during normal operation. `QLOOP` 's own `LDD #0 / STD STATE` , run at the top of the per-line loop, was a third, redundant reset that fired every time `QUERY` read a new line - including lines in the middle of an still-open colon definition, regardless of whether `;` had actually been reached. `TRUE` (and everything after it on a continuation line) got interpreted and, for anything with a stack effect, executed instead of compiled - `TRUE` pushing `TRUEV` onto the data stack instead of being compiled, leaving a stray cell `CSP` correctly caught as a mismatch at `;` . Confirmed the fix's placement carefully rather than just deleting the reset outright: `QUIT` is only re-entered on cold boot or an uncaught error routing back through `ABORT` - confirmed ordinary successful lines loop back to `QLOOP` directly, never `QUIT` - so moving the reset to run once at `QUIT` (rather than removing it entirely) still correctly forces interpret state exactly when ANS's own `QUIT` semantics call for it (including recovering from an error that aborts an unfinished colon definition, which deleting the reset outright would have left permanently stuck in compile mode), just not on every single line of an otherwise-uninterrupted session. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found via MAME testing and fixed, surfaced directly by the prior QLOOP/STATE fix: `QOK` skipped the `CR` echo together with the "ok" message whenever `STATE` was nonzero, silently dropping the line ending for every line of a multi-line colon definition after the first.** Once the QLOOP fix let multi-line colon definitions actually compile successfully, the echoed source no longer resembled what was typed - every continuation line ran together onto one visual line, since the terminal never saw the `CR` the user had genuinely pressed. Traced to `QOK: LDD STATE / BNE QLOOP` running *before* `JSR CRW` - while compiling, the branch away skipped both the `CR` and the "ok" message together, when only the "ok" message is actually supposed to be conditional (correctly not shown mid-definition). The error path just above `QOK` already got this right (unconditional `JSR CRW` before printing the error message) - only the success path had the bug. Fixed by moving `JSR CRW` to run unconditionally, first, with the `STATE` check (and the "ok" message it guards) coming after. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Real bug found via MAME debugger and fixed: `FILL` corrupted its own remaining-byte count on the very first iteration, running far past the requested length.** Same register-clobber class as `CCALL` 's original bug: `FILLOOP` loaded `D` with the count, then `LDA FILLCHR` (needed for the fill byte) overwrote `A` - `D` 's high byte - before `SUBD #1` decremented what it believed was still the true count. The high byte took on the fill character's value instead, so the count almost never reached zero at the intended point. Fixed by keeping the count in `Y` instead of round-tripping it through `D` /memory each iteration - `Y` is untouched by loading the fill character into `A` , so the conflict is structurally impossible now, not just avoided by careful ordering. Also addressed the redundant scratch-register usage flagged alongside the bug report: the target address is now kept directly in `X` (via `TFR D,X`) rather than round-tripping through `FILLADDR` first, matching the same treatment given to the count. `FILLCNT` / `FILLADDR` (aliases for the shared `MVCNT` / `MVDST` scratch cells) became genuinely unused once the loop no longer touches memory for them - confirmed via search before removing, and removed along with their comment-block entries, consistent with how `DICTTOP` was handled earlier this session. `MVCNT` / `MVDST` / `MVSRC` themselves are untouched, since `HSLEN` / `HSADDR` / `MRESULT` / `FILLCHR` still alias them for other routines. `ERASEW` (which `JMP` s into `FILLW`) confirmed unaffected, since only the internal loop changed, not the external calling convention. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Real bug found via MAME debugger and fixed: `HEXBYTE` (used by `DUMP` 's hex column) read from `MSCR+1` instead of `MSCR` , formatting whatever stale byte**

happened to be at the never- written address instead of the byte actually passed in. `STB MSCR` writes one byte, at `MSCR` itself; both subsequent reads (`LDB MSCR+1` , once for the high nibble via four `LSRB` s, once for the low nibble via `ANDB #$0F`) used the wrong address - a cell `HEXBYTE` never writes at all, so both nibbles were derived from stale scratch content rather than the real value. Reported via `MOVE + DUMP` : a fill character of `$7B` showed as `$03` in the hex column, while the ASCII column (a separate code path, unaffected) correctly showed `}` . Confirmed `MSCR` is shared, general-purpose scratch used pervasively elsewhere via full 16-bit `STD` / `LDD` - `HEXBYTE` 's single-byte `STB` / `LDB` pattern was the outlier, and nothing else depends on `MSCR+1` holding anything meaningful at the point `HEXBYTE` runs. Fixed by changing both `LDB MSCR+1` occurrences to `LDB MSCR` , reading from where the byte was actually stored. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Formatting change, not a bug fix: `DUMP` now emits a leading `CR` before its first line of output, matching the trailing `CR` `DULEND` already emits after every line (including the last).** Requested for consistent vertical alignment - without a leading `CR` , the first line started wherever the cursor already was (e.g. right after the echoed command itself), while every subsequent line correctly started at column 0 via the existing trailing `CR` . Added `JSR CRW` as the first thing `DUMPW` does, right after popping its two arguments and before the per-line loop begins. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed: `CMOVE` never terminated - same register-clobber class as `FILL` 's bug earlier this session.** `CMVLOOP` loaded `D` with the count, then `LDA ,X+` (needed to fetch the byte being copied) clobbered `A` - `D` 's high byte - before `SUBD #1` decremented what it believed was still the true count, so the terminating condition almost never fired. Checked the two adjacent, structurally similar routines rather than assume they shared the bug: `CMOVEGT` (the reverse-direction, overlap-safe variant) is genuinely unaffected, since its own copy (`LDA ,X`) happens *before* `LDD MVCNT` reloads the count fresh from memory each iteration, rather than after - the clobber happens, but the count is correctly reloaded afterward, overwriting it, not corrupted by it. `MOVEW` has no copy loop of its own at all - it's purely a dispatcher choosing `CMOVEW` or `CMOVEGT` based on overlap direction, so it's correct automatically once `CMOVEW` is. Fixed `CMOVEW` differently than `FILL` : unlike `FILL` (which only needed one address, leaving a register free for the count), `CMOVEW` already commits both `X` (source) and `Y` (destination) to the two addresses, leaving no spare 16-bit register to hold the count in in the same way. Fixed instead by reordering - decrement and store the count while `D` still holds the true value, before the byte copy is free to clobber `A` .

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real design gap found via MAME testing and fixed: `SUBSTITUTE` never returned the ANS-required substitution count, and its core algorithm was a plain substring search-and-replace on the bare registered name rather than the ANS `%`-delimited scan.** Confirmed against the actual spec (forth-standard.org/standard/string/SUBSTITUTE and …/REPLACES, both fetched and read in full): `SUBSTITUTE` must scan for text between `%` (ASCII `$25`) delimiter pairs specifically - `%%` collapses to a single `%` (count unchanged); a name matching the `REPLACES` registration has the *entire* `%name%` span, delimiters included, replaced by the substitution text (count incremented); a non-matching name is passed through unchanged, delimiters and all (count unchanged); a trailing `%` with no closing delimiter passes the residue through unchanged. Rewrote `SUBSTITUTEW` completely to implement this scan, no longer calling `SEARCHW` at all - confirmed `SEARCHW` still has its own dictionary header (`H_SEARCHW`) and remains a legitimate standalone word (`SEARCH`) independent of this change. Reused the existing `COMPAREW` for the name-matching step rather than writing a new comparison loop. `GLOBALS` is fully packed (256/256 bytes, confirmed) - the rewrite reuses `MSCR` / `MSCR2` / `MSCR3` / `MSCR4` (confirmed untouched by `SUBCOPY`) for the new algorithm's scan state rather than adding dedicated cells. Also fixed the missing third return value - `SUBSTITUTEW` now pushes the substitution count (`0` or positive on success) alongside the destination address and length, matching `( addr1 len1 addr2 len2 -- addr2 len3 n )`. The destination-overflow behavior (`THROW -1`) was left unchanged - the ANS spec explicitly permits throwing for negative-`n` cases in the standard THROW-code table.

```
**Separately identified, and deliberately NOT fixed on request:**
`S"`, when interpreted (not compiled - i.e. typed directly at
the prompt rather than inside a colon definition), always
writes into the same fixed, shared buffer (`SIBUF`), returning
that same address every time. `REPLACES` only stores the
addresses it's given, not copies of the text - so two `S"`
calls used to register a name/value pair (and a third `S"` for
`SUBSTITUTE`'s own source string) all silently overwrite the
same buffer, corrupting earlier registrations before
`SUBSTITUTE` ever runs. Traced precisely enough to reproduce a
reported garbled MAME output (`Dancinging, %girl%!`) by hand,
confirming the exact mechanism. User's explicit decision: don't
add dedicated copy-on-register storage to `REPLACES` - instead,
wrap each string in its own colon definition (`: name S"
text" ;`) for stable, independent storage, and document this
requirement in the glossary instead of changing the code. Done -
see the `REPLACES`/`SUBSTITUTE` glossary entries in
```

```
ClaudeForth.docx, now also corrected to match the real ANS stack
effects and behavior rather than the previous, inaccurate
descriptions.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly. Given the size
of the rewrite, long branches (`LBRA`/`LBHS`/`LBNE`) were used
liberally for loop-backs and larger jumps as a safety margin.
**MAME-CONFIRMED** (single and double substitution both
correct: `S" Dancing, %girl%!" poem 80 SUBSTITUTE` -> "Dancing,
Agnetha!"; `S" Two dancing girls- %girl%! %girl%!" poem 80
SUBSTITUTE` -> "Two dancing girls- Agnetha! Agnetha!", with
`.S` showing `2` as the substitution count) - real assembly and
execution confirm the long-branch margin was sufficient and the
`%`-delimiter algorithm itself is correct.
```

- **Verification task, requested and completed: confirmed PAD's current offset satisfies all three ANS transient-region minimums (forth-standard.org/standard/usage, section 3.3.3.6), added named equates documenting them, and redesigned S" to use PAD instead of a fixed, retired SIBUF.** Verification result: already satisfied, no numeric adjustment required. - PAD's own scratch region (>=84 chars): `PADW` computes `PAD = CODEHERE + 84` - meets the minimum exactly, with the usual ANS-expected transient/dynamic behavior (moves as CODEHERE advances). - WORD's transient region (>=33 chars): `WORDBUF` spans `$01DA` to `$01FA` (up to where the now-retired `SIBUF` began) - computed precisely via address subtraction (`$01FB - $01DA = 33`), meeting the minimum exactly. Unrelated to the PAD offset - this system's `WORD` uses its own separate, fixed buffer rather than HERE-relative storage. - Pictured numeric output buffer (>= (2*16)+2 = 34 chars): traced `LTNUM` (" <#") and `HOLD` precisely - `HLD` anchors to `PADW` 's own return value, and `HOLD` decrements *before* storing, so this buffer grows downward from PAD into the CODEHERE-to-PAD gap, not into PAD's own region above it. 34 <= 84, comfortably satisfied with 50 bytes of margin.

```
Added `PADMINSIZE`/`WORDMINSIZE`/`HOLDMINSIZE`/`PADOFFSET` as
named, documented equates, replacing the bare `84` that used to
sit directly inside `PADW`.

Redesigned `S"`'s interpreted-mode path (`SQINTERP`) to write
into PAD (computed fresh via `PADW` on every call) instead of
the old, fixed `SIBUF` - giving interpreted `S"` access to
PAD's full 84-character region instead of the previous fixed
32-character limit. Confirmed `SIBUF` was used exclusively by
```

`S"` (matching the user's own stated assumption, verified by search) before retiring its `EQU` definition; left its old 32-byte address range ($01FB-$021A) unclaimed rather than shifting `APPVARS` down to reclaim it, to avoid any risk to the rest of this carefully-verified memory map for the sake of 32 bytes on a system with headroom to spare.

Difficulties check, as requested: confirmed no conflict with the pictured-numeric-output buffer (opposite growth directions from the same PAD anchor - traced precisely, they don't overlap). Confirmed a genuine, expected trade-off instead: per ANS's own allowance ("non-standard words... may use PAD, but such use shall be documented"), PAD is now shared between the user's own general-purpose use and interpreted S" - using one right after the other will overwrite one with the other, same as any other transient-region interaction the standard itself anticipates. Also identified and documented a subtler point: PAD redesign does NOT eliminate the need for the user's own colon-definition-wrapping workaround (established two turns ago) - PAD's address only changes when something is compiled or allocated between calls, so two interpreted S" calls in a row (e.g. REPLACES's two arguments) still collide, just via PAD instead of SIBUF. Compiled S" (inside a colon definition) never touches PAD at all, writing directly into the definition's own permanent storage instead - this remains the correct approach for anything needing more than one string to coexist. Confirmed `SIBUF` referenced nowhere else in the file before retiring it.

Glossary entries for `PAD`, `S"`, and `REPLACES` updated to match - the `REPLACES` entry's prior `SIBUF`-specific wording (from the previous SUBSTITUTE fix) was stale and has been corrected to describe the current PAD-based mechanism instead.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-exact split-file reassembly; both updated Assembler Source appendix subsections (8.1 Memory Map and 8.20 String Words) and all three updated glossary entries confirmed present in the rendered PDF. **MAME-CONFIRMED** as part of the full substitution chain below - the PAD-based interpreted-mode S" storage this entry introduced is exercised correctly by every scenario in that confirmation.

- **Confirmed and fixed at the user’s request: WORD had its own, separate, fixed 31-character-capped buffer (WORDBUF), entirely unrelated to CODEHERE or PAD - the PAD-based S” redesign two turns ago didn’t help long S” strings because**

WORD truncates during parsing, an earlier stage than either S"'s storage path. Redesigned WORD to scan directly into the CODEHERE-to- PAD gap instead, matching the traditional fig-Forth layout (WORD's buffer at the CODEHERE end, the pictured numeric output buffer at the PAD end, growing toward each other) - added `WORDMAXCHARS` as a named constant reserving `HOLDMINSIZE` bytes at the PAD end for that buffer, so the two don't collide even though ANS itself would permit the overlap (3.3.3.6). `WORDBUF` confirmed genuinely unused before retiring it, same treatment as `SIBUF` 's retirement two turns ago.

```
**Serious collision bug caught and fixed while verifying this
redesign, before delivery - not reported by the user, found by
tracing the change's own consequences.** `SQUOTE` (S"),
`DOTQUOTE` (."), and `AQSTOK` (ABORT") all compile a 3-byte
runtime trampoline (`JSR CCALL`) directly at `CODEHERE` *after*
calling `WORD` but *before* copying the parsed text - with
`WORD` now writing that same text at `CODEHERE` too, the
trampoline would overwrite the first bytes of the very text
being staged, before the copy loop ever ran. Checked every
other caller of `WORD` in the file systematically (`TO`, `IS`,
`ACTION-OF`, the main interpreter loop, `'`, `POSTPONE`,
`[']`, `CHAR`, `[CHAR]`, `(` ) - confirmed none of them share
this pattern, since they either resolve to an xt via `FIND`
immediately (never needing the raw text again) or extract a
single character into a register before any compilation runs.
`HEADER` (used by every defining word) is also unaffected,
since it writes into `DPHERE`, a completely separate region
from `CODEHERE`. Fixed all three affected words identically:
reserve 3 bytes ahead of `CODEHERE` before calling `WORD`, so
the parsed text naturally lands exactly where it needs to end
up, with the trampoline safely compiled into the reserved gap
in front of it rather than on top of it - the existing copy
loop becomes a harmless no-op (reading and writing the same
address) rather than needing to move anything.

Caught a further, related overflow while finishing this: the
3-byte reservation itself pushes the compiled path's text 3
bytes further into the CODEHERE-to-PAD gap than plain WORD-
parsing does (e.g. a dictionary name for FIND) - the original
`WORDMAXCHARS=49` would have let the worst case (S"/./ABORT",
at the cap) overflow 3 bytes into the pictured-numeric buffer's
reserved space. Corrected to `WORDMAXCHARS=46`, sized uniformly
for the worst case across all callers rather than tracking a
different effective limit per caller - simpler and safer.

Also caught and fixed a transcription error in my own edit
```

before it was ever delivered: while inserting an explanatory comment into WORD's scan loop, the `BEQ ENDW` branch instruction immediately following the length-cap comparison was accidentally dropped, which would have made the length check compare but never actually branch - re-verified against the live file and corrected before proceeding further.

Glossary entries for `WORD`, `S"`, `."`, and `ABORT"` updated to match - `S"`'s entry from the previous turn specifically is corrected here, since its claimed "up to PADMINSIZE (84 characters)" limit was always inaccurate: WORD's own separate cap (31 at the time, now 46) was still the binding constraint, never 84.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-exact split-file reassembly; all four updated Assembler Source appendix subsections (8.1 Memory Map, 8.10 Outer Interpreter, 8.14 Compiling Words, 8.20 String Words) and all four updated glossary entries confirmed present in the rendered PDF.

MAME-CONFIRMED across every scenario that mattered, including the highest-risk one: `: expand S" Two dancing girls- %girl%! %girl%!" poem 80 SUBSTITUTE DROP SPACE TYPE ;` then `expand` -> "Two dancing girls- Agnetha! Agnetha!" - this specifically exercises the compiled-mode collision fix in `SQUOTE` (the reserve-3-bytes-before-calling-WORD pattern), confirming the runtime trampoline and the parsed text no longer overwrite each other. Also confirmed: the 34-character (and longer, 36-character) strings that originally exposed WORD's old 31-character cap now parse in full, in both interpreted (`S" ..." poem 80 SUBSTITUTE`) and colon-definition-wrapped (`: template S" ..." ;` then `template poem 80 SUBSTITUTE`) forms, with correct output in every case.

`DOTQUOTE` (`."`) separately MAME-confirmed too - its own reserve-3-bytes fix, applied identically to SQUOTE's but as a distinct edit, exercised independently: `: .message ." <text> " ;` then `.message` types the text correctly, unmodified.

`AQSTOK` (`ABORT"`) also separately MAME-confirmed, completing independent confirmation of all three fixed words: `: over18 18 < ABORT" Must be > 18" ;` - `3 over18` correctly types the message and throws -2 (the true-flag path); `18 over18` correctly falls through with no message and no throw (the false-flag path). Both branches of ABORT"'s own conditional

```
logic exercised, not just the message-compilation mechanics
shared with SQUOTE/DOTQUOTE.
```

- **Real, significant bug found via MAME testing and fixed: `UNESCAPE` 's argument popping was completely wrong - only one of its three arguments was ever popped, and even that one went into the wrong variable.** ANS signature is `( c-addr1 u1 c-addr2 -- c-addr2 u2 )` . The code did `PULU D / STD UESRCLEN` (storing the popped `c-addr2` , the real destination address, into the *source-length* variable), then `LDD ,U` - a peek, not a pop - into *both* `UEADDR` and `UEDST` (misreading `u1` , the real source length, as an address, and never popping it off the stack at all). `c-addr1` , the actual source address, was never touched - left sitting on the stack the entire time. Net effect: `UEADDR` / `UEDST` both ended up holding a small length value misread as an address, the copy loop read and wrote through whatever garbage that pointed at (low memory, within GLOBALS itself, given typical string lengths), and the real source/dest addresses were either misfiled or left stranded on the stack - matching the reported symptom precisely (two addresses left on the stack, no real count, buffer untouched). Also fixed `UEDONE` 's return: it used to do `LDD UEOUTLEN / STD ,U` , overwriting whatever was left on top of the stack rather than pushing both required return values; now correctly pushes `c-addr2` (destination address) then `u2` (actual unescaped length), matching the ANS stack effect. Confirmed `UNESCAPEW` is referenced nowhere else in the file before applying the fix. The escape-processing loop itself (the `\n` / `\t` / `\\` / `\"` handling, including the lone-trailing-backslash fix from earlier this session) was untouched and unaffected - it operates correctly once its input variables are actually populated correctly. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Major finding, well beyond the prior turn's argument-order fix: `UNESCAPE` 's entire algorithm was wrong from the start, not just its argument popping.** After the previous fix corrected the broken argument order, MAME testing showed doubling a single `%` wasn't happening at all, and the returned count didn't grow to account for it. Investigated against the actual spec rather than assuming a smaller bug within the existing logic - fetched both forth-standard.org/standard/string/UNESCAPE and the complang.tuwien.ac.at ANS Forth reference text, which includes the canonical reference implementation. Confirmed: `UNESCAPE` has nothing to do with backslash escape sequences (`\n` , `\t` , `\\` , `\"`) at all - the entire pre-existing implementation (predating this session) was decoding a plausible-sounding but entirely wrong operation. The real ANS `UNESCAPE` replaces every `%` character with two `%` characters and passes everything else through unchanged, specifically so that a literal `%` in text survives an eventual `SUBSTITUTE` pass unchanged ("If you pass a string through UNESCAPE and

then SUBSTITUTE, you get the original string"). Replaced the entire routine body accordingly - confirmed the old backslash-handling labels (`UELASTBS` / `UEPLAIN` / `UECKT` / `UECKBS` / `UEEMIT`) were internal to this one routine before removing them. New output length is computed directly as the final write pointer minus the starting destination address, correct regardless of how many `%` characters were doubled, rather than incrementally tracked per-character the way the old (wrong) loop did it. Manually traced the new algorithm against the ANS reference's own test case (`"aaa%bbb" UNESCAPE` should equal `"aaa%%bbb"`) before delivery - confirmed matching exactly, including the length (7 characters in, 8 out). No destination-buffer-overflow check added, matching the ANS reference implementation, which also has none (an ambiguous condition per the standard, not a required error case). Glossary entry corrected completely - the prior entry's stack effect, description, and even its "result length is always <= input length" side-effect claim were all wrong, describing the old, incorrect algorithm rather than the real one (output can now be longer than input, by design). Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly; updated Assembler Source appendix subsection (8.20 String Words) and the corrected glossary entry both confirmed present in the rendered PDF. **MAME-CONFIRMED**: correctly escapes `%` characters as required.

- **Feature gap confirmed and addressed: the text interpreter had no support for ANS's double-number input convention (trailing '.') - "123." was silently treated identically to "123", leaving no way to get a genuine ud1 onto the stack for use with # and friends.** Confirmed against the precise spec text (forth-standard.org/standard/usage#usage:numbers, 3.4.1.3: "When the text interpreter processes a number... that is immediately followed by a decimal point... the text interpreter shall convert it to a double-cell number. For example, entering DECIMAL 1234 leaves the single-cell number 1234 on the stack, and entering DECIMAL 1234. leaves the double-cell number 1234 0 on the stack."). Traced `NUMBERQ` and found it already accumulated a full 32-bit value internally (`UDHI` : `UDLO` , via `NUMLOOP`) for every number parsed, single or not - but only ever returned the low cell, discarding `UDHI` unconditionally, and had no detection of a trailing '.' at all. Also found the existing negation only negated the low 16 bits - harmless while only ever returning a single cell, but wrong once a genuine 32-bit value needed returning.

```
Redesigned `NUMBERQ`: detects a trailing '.' early (excluding
it from the digit count before `NUMLOOP` runs), performs full
32-bit two's-complement negation (low cell negated first, any
carry out of that addition propagated into the high cell,
manually traced against both a normal case and the zero-wraps-
to-zero edge case before delivery), and re-derives the double-
```

vs-single decision independently at the very end of the routine by re-reading from `CADDR` and the original count - both confirmed unchanged throughout the routine's entire execution, including through `NUMLOOP`/`UDMULADD` - rather than remembering the decision in a new persistent flag. This was a deliberate choice: `GLOBALS` is fully packed at 256/256 bytes (the 6809's own direct-page addressing limit, not an arbitrary number), so avoiding a new flag byte avoided touching that boundary at all. Return convention on success now distinguishes double (pushes low, high, then success code `1`) from single (pushes value, then the original `-1`, completely unchanged) - chosen so `TRYNUM` doesn't need a separate flag either, and so single-number behavior is provably identical to before by construction, not just by testing.

Updated `TRYNUM` to handle both codes: on double success while compiling, `UDHI` (on top of `U`) is stashed in `MSCR4` so `UDLO` can be compiled as the first of two `LIT` literals, then `UDHI` as the second - ensuring they execute in the order needed to land correctly on the stack at runtime (low deep, high on top, per 3.1.4.1's "the cell containing the most significant part... shall be above the cell containing the least significant part"). While interpreting, both cells are already correctly placed by `NUMBERQ` and need no further handling.

Confirmed `TONUMBER` (`>NUMBER`) is untouched - it calls the same shared `NUMLOOP`, but neither its own code nor `NUMLOOP` itself needed any change; only `NUMBERQ`'s own wrapper logic and `TRYNUM`'s consumption of it were modified.

Manually traced three cases by hand before delivery, beyond structural verification alone: "123." (interpreted) -> [123, 0] correctly, matching the ANS example exactly; "123" (no dot) -> [123, -1] internally, correctly falling through to the original, unchanged single-cell path, confirming backward compatibility; "-123." -> UDHI=$FFFF/UDLO=$FF85, confirmed by hand to equal $FFFFFF85 (i.e. -123 as a 32-bit value).

Glossary entries for `<#` and `#` updated with a note on how to get a proper ud1 onto the stack from typed input, since that was the specific gap originally reported.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-exact split-file reassembly; updated Assembler Source appendix subsection (8.10 Outer Interpreter) and both updated glossary entries confirmed present in the

```
rendered PDF. Not yet confirmed via MAME - this is a
substantial, multi-path change (single/double x interpret/
compile x positive/negative all interact) and deserves thorough
real-hardware testing across that full matrix before being
considered settled.
```

- **Two real bugs found via a very thorough MAME test matrix (interpret and compile mode, both signs, single and double, cumulative `.s` traces across many lines) and fixed - both in the 32-bit negation logic delivered last turn for double-number input, and both only visible on negative doubles specifically.** Positive cases (`123. -> 0 123` , `123456. -> 1 -7616`) were already confirmed correct by the trace, matching the design exactly - the negation path (which only positive numbers skip entirely) was where the actual problems were.

```
**Bug 1**: the 6809's `COM` instruction (one's complement)
unconditionally sets the carry flag to 1, regardless of its
operand - a documented CPU quirk. The negation code computed
the real carry from the low cell's `ADDD #1`, but then ran
`COMA`/`COMB` on the high cell before checking that carry - and
`COMB` silently overwrote it with its own, always-set value, so
the intended "only propagate carry into the high cell when the
low cell actually overflowed" check never worked. `-123.`
returned high cell `0` instead of `-1`; `-123456.` returned `-1`
instead of `-2` - both exactly matching "always add 1"
regardless of the true carry, not the intended conditional
behavior. Fixed by saving the true carry via `PSHS CC`
immediately after the low-cell addition, before the high-cell
`COM` can destroy it, then `PULS CC` to restore it right before
the branch that depends on it.

**Bug 2, only exposed by fixing Bug 1**: with the carry check
now actually working, the branch it guarded (`BCC`) turned out
to jump all the way past `STD UDHI` itself when there's no
carry - not just past the `+1`. The complemented high-cell value
was computed in D but never actually stored back to UDHI in
that case, leaving it at its stale, pre-negation value. This was
completely masked by Bug 1 in practice - since carry was always
corrupted to "set," the branch was never actually taken, so the
store always ran (with the wrong value, per Bug 1, but it did
run). Fixing Bug 1 alone would have made this second, previously
-latent bug live and produced a different set of wrong answers.
Restructured so the store always runs, with the conditional
`+1` applied before it rather than gating whether it happens at
all.
```

```
Manually re-traced both negative-double test cases against the
fully corrected code before delivery: `-123.` -> carry clear on
the low cell, branch skips only the `+1`, raw complement `$FFFF`
gets stored -> high cell `-1`, correct. `-123456.` -> same
pattern, raw complement `$FFFE` stored -> high cell `-2`,
correct (matches the true 32-bit value `$FFFE1DC0` computed by
hand). Also re-traced the carry-set path (negating 0, as a
sanity check) to confirm that branch still correctly falls
through to the `+1` before storing.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly; updated
Assembler Source appendix subsection (8.10 Outer Interpreter)
confirmed present in the rendered PDF. Glossary entries (`<#`,
`#`) needed no further changes - this fix doesn't alter the
stack effect or usage, only the underlying correctness.

**MAME-CONFIRMED**, both interpret and compile mode, across the
full t0-t7 matrix: `-123.` -> high `-1` / low `-123`; `-123456.`
-> high `-2` / low `7616` - both matching the hand-traced
prediction from the prior turn exactly (high `$FFFF`/`$FFFE`
respectively, confirmed independently against `$FFFFFF85` and
`$FFFE1DC0`). Positive cases (`123.` -> `0 123`, `123456.` ->
`1 -7616`) and single-cell cases (unaffected by either bug, per
the negation code being skipped entirely) also confirmed
correct. Both the carry-corruption bug and the masked missing-
store bug are now verified fixed on real hardware, not just by
static hand-tracing.
```

- **Two explicit, requested adjustments applied: `BASECODE` and `BASEDICT` origins shifted down $40 (64 bytes) each, to make room for growth since they were last positioned; `UNITTESTS` set to 1 (excluded) since unit tests don't work yet and resolution has been postponed.** `BASECODE` : `$E02A` -> `$DFEA` . `BASEDICT` : `$D83F` -> `$D7FF` . Applied exactly as requested, without independently recomputing the resulting gaps/overlaps against `INITCODE` and each other - this region's comments have historically tracked those in precise byte counts, but doing so accurately requires real assembled sizes (`BASECODESIZE` / `BASEDICTSIZE` , computed via `*-start` at each section's end), which structural verification here cannot determine - it checks label integrity and dictionary-chain structure, not location-counter arithmetic. The comments at both `EQU` s now say this explicitly rather than presenting a static estimate as if it were a real assembled count - confirm the resulting gaps/overlaps on real assembly.

```
Verified: zero duplicate symbols, dictionary chain still 224
```

```
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations (checked with both `UNITTESTS` values regardless
of the file's own new default); byte-exact split-file
reassembly.
```

- **Real bug found via MAME debugger and fixed: `S>D` performed no sign extension at all - the same register-flag class of bug already documented once this session in `TRYNUM`.** `PULU` does not affect condition codes on genuine 6809 - `STOD` did `PULU D` then immediately `BPL SDPOS`, testing whatever flags an unrelated, earlier instruction happened to leave set, not the sign of the value just popped. `-123 S>D` returned high cell `0` instead of `-1`. Fixed by adding an explicit `TSTA` (testing D's high byte, whose bit 7 reflects the full 16-bit value's sign) immediately before the branch that depends on it. Confirmed `STOD` is referenced only via its own dictionary header before applying the fix - nothing else calls it directly. Checked the neighboring routines (`DTOS`, `DMAXW`) for the same pattern rather than assume they're clean by proximity alone - both confirmed unaffected: `DTOS` has no branch at all, and `DMAXW` already uses an explicit `CMPD` before its own branch. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Real bug found via MAME testing and fixed: `SIGN` had the same `PULU`-doesn't-set-flags defect already found in `STOD` - plus a separate, genuine usage-pattern issue in the reported test definition, diagnosed but not a code bug.** Reported via `: format DUP ABS S>D <# #S SIGN #> TYPE ;` never printing a minus sign for either sign of input.

```
**Code bug**: `SIGN` did `PULU D` then immediately `BPL
SIGNDONE` - `PULU` doesn't affect condition codes on genuine
6809, so the branch tested whatever flags an unrelated, earlier
instruction happened to leave set, not the sign of the value
just popped. Fixed with the same `TSTA` pattern used for `STOD`.
Checked all four existing internal callers (`DOT`/`DOTR`/
`DDOT`/`DDOTR`, i.e. `.`/`.R`/`D.`/`D.R`) before concluding
anything about their own correctness - all four turned out to
be accidentally unaffected, since each runs `LDD SAVEN` (a
flag-setting load of the true signed value) immediately before
`PSHU D`/`JSR SIGN`, and `PSHU` doesn't disturb flags. This was
fragile, caller-side luck rather than `SIGN` being correct on
its own - confirmed precisely by the fact that a direct,
standalone use (the reported test word) had no such protection
and failed outright.

**Separate, non-code issue identified in the reported
definition itself**: even with `SIGN` fixed, `DUP ABS S>D <#
```

```
#S SIGN #>` would still never add a minus sign, traced
precisely - `#S` fully consumes its `ud` down to `0 0`, so
whatever sits on top of the stack immediately after `#S` is
always `0`, genuinely not negative, regardless of the original
number's sign. The true signed value (left underneath by `DUP
ABS`) never reaches the top of the stack in that sequence at
all. Standard idiom needs the signed value stashed on the
return stack and restored right before `SIGN`: `DUP >R ABS
S>D <# #S R> SIGN #>`. Not a defect - flagged so the fix to
`SIGN` itself isn't mistaken for resolving this specific
definition's own behavior too.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly. Not yet
confirmed via MAME.
```

- **Investigated, no code change made: an apparent `CATCH` success-path anomaly (`0` `process` , i.e. no `THROW` induced, showing nothing on `.S` instead of the expected `0`) turned out not to reproduce on retest, most likely a MAME single-stepping artifact rather than a genuine bug.** Traced `CATCH` 's success path precisely (the `LEAS 2,S` / `PULS D` / `STD HANDLER` / `LDD #0` / `PSHU D` / `RTS` sequence after a normal, non-throwing return) and separately `0BRANCH` (`ZBRANCH` , the specific code path taken since `0<> of 0` is false in the reported test, meaning `IF` 's body is skipped) - both confirmed structurally balanced and correct via static tracing, with no bug found on paper. User's own leading hypothesis, given this: MAME's 6809 core "jumping into other routines on single-stepping instructions that were not branches/subroutines" - i.e. a debugger artifact from stepping through raw memory, not a defect in the actual compiled code. Consistent with the static trace finding nothing wrong and the issue not reproducing on a subsequent, real (non-single-stepped) run - a genuine code bug wouldn't self- resolve between otherwise-identical test runs. Confirmed working via a complete, realistic follow-up test (`loge` / `process` , chaining `CATCH` success and failure paths together with `SIGN` and `HOLDS` , both fixed earlier this session): `0 process -> OK.` (success path correct); `5 process -> Err: -4000` (failure path correct, `SIGN` / `HOLDS` interaction correct end to end). No source change was needed or made.
- **Real bug found via MAME testing and fixed: `(` (`LPAREN`) left a stray address on the stack after every comment.** `WORD` always pushes a `c-addr` (the parsed text's address) at the end of its own execution, per its established calling convention - `LPAREN` only ever needed `WORD` 's side effect of advancing `>IN` past the closing `)` , never the address itself, but never popped it either, going straight to `RTS` . Explained both reported symptoms precisely: a stray value left on the stack after a comment in interpret mode (`(`

`Math for */ ) .S -> 28888`, a plausible `CODEHERE` value within the `APPCODE` region, matching what `WORD` 's redesign earlier this session would push), and a `-22` CSP mismatch for any colon definition containing a comment, since the leftover throws off the compile-time stack-depth check between `:` and `;`. Confirmed `LPAREN` is referenced only via its own dictionary header, and confirmed there's no `.(` (a related, commonly-paired comment word) in this system that might share the same pattern - `(` is the only comment word affected. Fixed with a single `PULU D` to discard the address, immediately after `JSR WORD` returns. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. **MAME- CONFIRMED**: `(` now works correctly.

```
`\` (`BACKSLASH`) separately tested alongside `(` as part of
this same comment-words sweep - confirmed already working
correctly, no code change needed.
```

- **Verification, no code change: confirmed `WORDLISTS` (16.3.2, Search-Order word set) already, correctly returns `false` from `ENVIRONMENT?`.** Checked the full `ENVTABLE` (four entries: `/COUNTED-STRING`, `MAX-N`, `MAX-U`, `ADDRESS-UNIT-BITS`) - `WORDLISTS` is genuinely absent, not an oversight, since this system doesn't implement the Search-Order word set at all. Confirmed against `ENVIRONMENT?` 's own spec (6.1.1345): "If the system treats the attribute as unknown, the returned flag is false" - falling through to `ENVNOTFOUND` already produces exactly that, correctly and completely, with no dispatcher work needed. Clarified the pre-existing comment above `ENVQUERY`, which previously lumped `WORDLISTS` together with `MAX-D` / `MAX-UD` / `FLOORED` as all equally "need[ing] dispatcher extensions not yet built" - that's true for `MAX-D` / `MAX-UD` (genuinely incomplete: both are double-cell values, but `ENVFOUND` only ever pushes one cell before the `TRUE` flag) but not for `WORDLISTS`, whose correct answer is simply "unknown," which absence from the table already provides. `FLOORED` kept separately tracked as its own, still-open item - unlike `WORDLISTS` it's a *core* environmental query, not tied to an unimplemented optional word set, so this system likely has a genuine, definite division behavior it could report - "false by omission" isn't necessarily the right answer for it the way it is for `WORDLISTS`. Not investigated further here; left explicitly open rather than silently dropped from tracking. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Two real bugs found via MAME testing and fixed in `ENVIRONMENT?` : `X` register clobbered across a `COMPAREW` call, and a wrong table value predating this - together explained the user's exact reported value precisely, not approximately.** Reported via `S" /COUNTED-STRING" ENVIRONMENT?` returning `-1 $4E4D` instead of the

expected `-1 255`.

```
**Root cause**: `COMPAREW` uses `X` as its own internal scratch
register (`LDX CMPA1`, then `LDA ,X+` advancing through the byte
-by-byte comparison) and never saves or restores it - any caller
relying on `X` surviving the call gets it clobbered. `ENVQUERY`
does exactly that: it needs `X` to still point at the current
table entry after `COMPAREW` returns, both for `ENVFOUND`'s own
`LDD 4,X` (reading the value field) and for advancing to the
next entry - but never saved it first. Traced precisely why this
produces `$4E4D` specifically, not just "some garbage": after a
successful match against `/COUNTED-STRING` (15 characters),
`COMPAREW` leaves `X` sitting exactly at the start of the next
table entry's own string, `"MAX-N"` - reading `LDD 4,X` from
there reads two bytes 4 characters into that string: `'N'`
(`$4E`) and `'M'` (`$4D`, the first character of the following
entry, `"MAX-U"`) - together, exactly `$4E4D`. Checked the only
other `COMPAREW` caller in the file (`SUBHASNAME`, from the
`SUBSTITUTE` rewrite earlier this session) before concluding
this was `ENVQUERY`-specific - confirmed unaffected, since it
does a fresh `LDX` immediately after its own `COMPAREW` call
rather than relying on the prior value surviving. Fixed with
`PSHS X`/`PULS X` bracketing the `JSR COMPAREW` call.

**Separate, pre-existing bug found and fixed alongside it**:
`/COUNTED-STRING`'s table value itself was `31`, not the ANS-
standard `255` - a counted string's maximum size is bounded by
its 1-byte count field (0-255), not 31, which looks like it was
mistakenly copied from an unrelated constraint (`WORD`'s own,
separate scan cap from an earlier version of this file, before
this session's `WORD` redesign). This alone wouldn't have
produced the user's exact reported value, but was independently
wrong regardless and worth fixing at the same time. Corrected to
`255`.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly. Not yet
confirmed via MAME.
```

- **Explicit, requested adjustment applied: `BASECODE` and `BASEDICT` origins shifted down a further $10 (16 bytes) each - the second such adjustment this session, needed for assembly to succeed as the codebase has continued to grow.**
  `BASECODE`: `$DFEA` -> `$DFDA` (originally `$E02A` before either shift). `BASEDICT`: `$D7FF` -> `$D7EF` (originally `$D83F`). Applied exactly as requested, same approach as the earlier

$40 shift - not independently recomputing the resulting gaps/overlaps against `INITCODE` and each other, since that requires real assembled section sizes this environment can't determine; confirm on real assembly. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **`/HOLD` added to `ENVTABLE` (previously genuinely absent, per the file's own pre-existing note), answering the design question of which of two candidate constants is correct: `HOLDMINSIZE` (34), not the larger `PADOFFSET` (84).** `HOLDMINSIZE` is specifically the portion of the `CODEHERE` -to- `PAD` gap *reserved* for the pictured numeric output buffer - the same amount `WORDMAXCHARS` deliberately holds back from `WORD` 's own use at the opposite end of that same shared gap. `PADOFFSET` is the gap's total width, most of which is actually earmarked for `WORD` 's parsing, not `HOLD` 's - reporting it would overstate what `HOLD` can safely use without risking collision with whatever `WORD` is doing in the same space. Confirmed `/HOLD` (the query string, with its leading slash) was genuinely absent from the file entirely before this - only the words `HOLD` / `HOLDS` themselves existed. The reported `255` result therefore couldn't have come from a legitimate table match; most likely a stale binary predating the `ENVQUERY X` -register fix from two turns ago, which would produce exactly this kind of unpredictable result for a query with no real match in the table. Updated both the top-of-file note and the `ENVQUERY` -local comment, which both referenced `/HOLD` as absent - now stale for `/HOLD` specifically; `/PAD` remains genuinely absent and separately tracked, not silently dropped. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. **MAME-CONFIRMED**: `/HOLD` now correctly returns `34` (`HOLDMINSIZE`) - also confirms the stale-binary explanation for the earlier `255` result was correct, not a lingering bug in the fix.

- **`/PAD` added to `ENVTABLE` , completing the pair the top-of- file note originally flagged as absent (`/HOLD` and `/PAD`) - both now present.** Answers `PADMINSIZE` (84) directly, per ANS's own `/PAD` meaning (3.3.3.6: "the size of the scratch area whose address is returned by PAD") - PAD's own region, growing upward from PAD itself, conceptually distinct from `HOLDMINSIZE` (which answers for a different region entirely, the downward-growing pictured-numeric buffer in the same `CODEHERE` -to- `PAD` gap, added last turn). `PADOFFSET` happens to equal `PADMINSIZE` numerically in this implementation (`PADOFFSET EQU PADMINSIZE`), but `/PAD` is answered with the conceptually correct constant rather than the coincidentally- equal one, matching the same discipline applied to `/HOLD` . Updated both the top-of-file note and the `ENVQUERY` -local comment, which both still referenced `/PAD` as absent - caught and fixed a duplication error introduced while editing the top-of-file comment (a leftover fragment

line duplicated “open-items checklist”) before it reached delivery. The DPHERE/CODEHERE/VARHERE boundary checks remain the one item still genuinely open in this area, kept explicitly tracked rather than dropped. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. **MAME-CONFIRMED**: `/PAD` now correctly returns `84` (`PADMINSIZE`).

- **Explicit, requested adjustment applied: `BASECODE` and `BASEDICT` origins shifted down a further $10 (16 bytes) each - the third such adjustment this session, needed for assembly to succeed as the codebase continues to grow.** `BASECODE` : `$DFDA` -> `$DFCA` (`$E02A` originally). `BASEDICT` : `$D7EF` -> `$D7DF` (`$D83F` originally). Applied exactly as requested, same approach as both earlier shifts this session - not independently recomputing the resulting gaps/overlaps against `INITCODE` and each other, since that requires real assembled section sizes this environment can’t determine; confirm on real assembly. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **`FLOORED` investigated and added to `ENVTABLE` , closing the one item explicitly left open two turns ago pending its own investigation.** Unlike `WORDLISTS` (correctly `false` by omission, since Search-Order is genuinely unimplemented), `FLOORED` is a *core* query - this system definitely has some division behavior, so “false by omission” wasn’t the right default. Investigated by tracing `DIVCOMMON` (shared by `/` , `MOD` , `/MOD`) against this system’s own `SM/REM` and `FM/MOD` implementations directly, rather than inferring from `DIVCOMMON` alone: `DIVCOMMON` restores the remainder’s sign from `DNSIGN` (the dividend’s own original sign) after dividing absolute values - confirmed byte-for-byte identical in structure to `SM/REM` ‘s own logic. `FM/MOD` , by contrast, has an explicit flooring-adjustment step (correcting the quotient and remainder when the quotient would be negative with a nonzero remainder) that `DIVCOMMON` does not have. Conclusion: this system’s primary division words use symmetric division, not floored. Added `FLOORED` to `ENVTABLE` with value `0` - a real, meaningful “recognized query, value false” result (two items: `0` then `TRUE`), distinct from the “unrecognized” single-item `false` fallthrough `WORDLISTS` correctly uses. Confirmed this doesn’t interfere with the table’s own terminator check, which reads each entry’s address field (offset 0), not its value field - a zero value is safe regardless of position in the table. Updated the `ENVQUERY` -local comment, which previously described `FLOORED` as still open. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split- file reassembly. **MAME-CONFIRMED**: `FLOORED` now correctly returns `0` (recognized, value false). Closes out the full `ENVIRONMENT?` sweep - `/COUNTED-STRING` , `/HOLD` , `/PAD` , `WORDLISTS` , and

FLOORED are all confirmed correct on real hardware.

- **MAX-CHAR (3.2.6, maximum value of any character in the implementation's character set) added to ENVTABLE with value 255, matching this system's 8-bit character set (ADDRESS-UNIT-BITS, already in the table, confirms this).** Confirmed MAX-CHAR was genuinely absent from the file entirely before this - same pattern as /HOLD / /PAD / FLOORED before they were added. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four SERIALPOLL x UNITTESTS combinations; byte-exact split-file reassembly. **MAME-CONFIRMED**: MAX-CHAR now correctly returns 255.

- **MAX-D / MAX-UD resolved via a genuine dispatcher extension, not just a table entry - the one item in this area explicitly flagged as needing real code work, not just data.** Both are double-cell values per their own ANS data type, but the original ENVFOUND path only ever pushed one cell before the TRUE flag - a table entry alone couldn't answer them correctly. Added a second table (ENVTABLE2 , 8-byte entries: addr/len/low/high, vs ENVTABLE 's 6-byte addr/len/value) and a matching second loop (ENV2START / ENV2LOOP / ENV2FOUND), rather than reworking the existing, already-tested single-cell path to handle both entry shapes at once - lower risk. The original table's "not found" path now tries the new table before finally giving up, rather than going straight to ENVNOTFOUND . MAX-D ($7FFFFFFF , low $FFFF /high $7FFF) and MAX-UD ($FFFFFFFF , low $FFFF /high $FFFF) added to the new table - low cell pushed first/deep, high cell second/ top, matching standard double-cell stack order (3.1.4.1). Updated the now-stale comment from two turns ago that described this as still needing dispatcher work. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four SERIALPOLL x UNITTESTS combinations; byte-exact split- file reassembly. **MAME-CONFIRMED**: MAX-D correctly returns -1 32767 -1 (low $FFFF , high $7FFF , then TRUE), and MAX-UD correctly returns -1 -1 -1 (low $FFFF , high $FFFF , then TRUE) - both independently confirmed on real hardware, not just structurally. The new double-cell dispatch path (ENVTABLE2 / ENV2LOOP / ENV2FOUND) works correctly for both entries it holds.

- **Explicit, requested adjustment applied: BASECODE and BASEDICT origins shifted down $40 (64 bytes) each - the fourth such adjustment this session, and larger than the three prior $10 shifts, matching the larger amount of new code added this turn (the MAX-D / MAX-UD double-cell dispatcher extension - a whole second table plus a matching lookup loop, not just a table row).** BASECODE : $DFCA -> $DF8A ($E02A originally). BASEDICT : $D7DF -> $D79F ($D83F originally). Applied exactly as requested, same approach as every prior shift this session - not independently recomputing the resulting gaps/overlaps against INITCODE and each other, since that requires real assembled section sizes this environment can't determine; confirm on real

assembly. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Explicit, requested adjustment applied: `BASECODE` and `BASEDICT` origins shifted down a further $20 (32 bytes) each - the fifth such adjustment this session. The prior $40 shift wasn't enough to resolve the collision, confirmed by trial and error against the real assembler.** `BASECODE`: `$DF8A` -> `$DF6A` (`$E02A` originally). `BASEDICT`: `$D79F` -> `$D77F` (`$D83F` originally). Applied exactly as requested, same approach as every prior shift this session - not independently recomputing the resulting gaps/overlaps against `INITCODE` and each other, since that requires real assembled section sizes this environment can't determine; the user's own real-assembler trial-and-error remains the actual source of truth for whether this resolves the collision, not a static estimate here. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **`BASECODE` and `BASEDICT` shifted down $80 (128 bytes) each - the sixth such adjustment this session. Both the prior $40 and $20 shifts proved insufficient - deliberately chose a larger jump this time (rather than another small increment) since two consecutive small adjustments had already failed to resolve the collision.** `BASECODE`: `$DF6A` -> `$DEEA` (`$E02A` originally). `BASEDICT`: `$D77F` -> `$D6FF` (`$D83F` originally). Same approach as every prior shift this session - not independently recomputing the resulting gaps/overlaps against `INITCODE` and each other, since that requires real assembled section sizes this environment can't determine; the user's own real-assembler trial-and-error remains the actual source of truth for whether this resolves the collision. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

```
**CONFIRMED RESOLVED** on real assembly - this shift cleared
the collision, with roughly 128 bytes of padding now free.
Consistent with the two prior insufficient shifts ($40 then
$20, summing to $60/96 bytes) having undershot the true
collision size, and this $80/128-byte shift landing with the
full amount to spare - the actual gap needed was evidently
somewhere between 96 and 128 bytes. Closes out this round of
memory-map adjustments; the next collision, whenever the
codebase grows enough to reintroduce one, will need its own
fresh trial and error rather than assuming this same margin
holds indefinitely.
```

- **`RETURN-STACK-CELLS` and `STACK-CELLS` added to `ENVTABLE`, completing the full set of**

MAME-testable environmental queries the user identified. Unlike every other entry this session, both use computed expressions tied to this system's own real stack-boundary constants (`RSTACK` , `DSTACK` , `CODETOP`) rather than hardcoded numbers - deliberate, given the memory map has shifted repeatedly this session (`BASECODE` / `BASEDICT` alone moved six times) and a hardcoded number would silently go stale on the next shift with nothing to catch it. `RETURN-STACK-CELLS = (RSTACK-DSTACK)/2` = 384 currently (RSTACK's own existing comment already confirms the occupied range is `$BD00 - RSTACK` , 768 bytes, and `DSTACK+1` is `$BD00` exactly per both regions' own comments confirming they're contiguous - the algebra simplifies to `RSTACK-DSTACK` directly). `STACK-CELLS = (DSTACK-CODETOP+1)/2` = 512 currently (`CODETOP` 's own comment: "code space ceiling (data stack begins here)"). Caught and simplified a redundant `-1+1` in the first draft of the `RETURN-STACK-CELLS` expression before delivery - algebraically identical but needlessly complex for an expression this environment can't test against a real assembler. Explicitly verified both expressions evaluate to exact integers (no truncation risk regardless of the assembler's own integer-division behavior) in addition to the standard structural checks, since FDB arithmetic isn't something the duplicate- symbol/dictionary-chain verification evaluates. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte- exact split-file reassembly. **MAME-CONFIRMED**: both correctly return `384` and `512` respectively - confirms the computed `FDB` expressions assembled and evaluated correctly on the real toolchain, not just in this environment's own arithmetic check. Closes out the full `ENVIRONMENT?` sweep - every entry in both `ENVTABLE` and `ENVTABLE2` is now independently confirmed correct on real hardware.

- **Documentation updated: title page duration entry, full Memory Map table rebuild, and a new Transient Region section - grounded in `forth6809_lst.txt` , a real assembler listing of the current source, uploaded this turn.** Confirmed the listing genuinely matches the current source before treating any of its numbers as authoritative - `BASECODE=$DEEA` , `UNITTESTS=1` , and this session's additions (`MAX-CHAR` , `MAX-D` , `RETURN-STACK-CELLS`) are all present in it, not a stale snapshot.

```
**Title page**: added "Manual MAME testing and bug-fixing
duration (guesstimate): 80 hours", matching the formatting of
the existing duration entries.

**Memory Map table, fully rebuilt** with real, assembler-
measured values rather than the old estimates: `BASECODE` =
8304 bytes (`BASECODESIZE`, was a stale 8110 estimate);
`BASEDICT` = 2027 bytes (`BASEDICTSIZE`). Discovered and
documented two previously-unverifiable facts directly from the
```

listing: `BASEDICTEND` equals `BASECODE` exactly (`$DEEA`) - zero gap, zero overlap - and `INITEND` equals `VECTORS` exactly (`$FFF0`) - also zero gap. The one real gap (`BASECODEEND` to `INITCODE`, 79 bytes) is explicitly `$FF`-filled by the assembler's own `FILL` directive, not an unaccounted collision. `APPCODE` row relabeled `APPCODE + Transient`, with a pointer to the new section, since the Transient region is a floating subregion within it rather than a fixed range of its own - chosen over a nested/indented row given the table's existing one-row-per-fixed-range structure. `APPDICT`'s upper bound corrected to `APPCODE-1` (20480 bytes; the prior `$6EA4`/20133 figure was stale, left over from an earlier `APPCODE` position before it moved to its current, "back to its original address" value). Retired `SIBUF`/`WORDBUF` rows removed (confirmed genuinely retired - their `EQU`s are commented out in the listing) and replaced with a single `(unclaimed)` row for the 65-byte range they used to occupy. "ROM Size Required" narrative rewritten with the corrected totals: 10418 bytes real ROM content (16+71+8304+2027), 5710 bytes free (~35%) - both now real, assembler-measured figures rather than the prior manual-count estimate the old text explicitly flagged as uncertain.

New "Transient Region" section, inserted at the same heading level as "Dictionary Entry Layout" (matching its style: `Normal` paragraph, bold run, no explicit size), immediately before it. Describes the region's function (floating, `CODEHERE`-relative, shared by `WORD`'s parsing buffer and the `HOLD` pictured-numeric-output buffer), size (`PADOFFSET`/`PADMINSIZE` = 84 bytes, `PAD = CODEHERE + 84`), and location, then a new sub-table (Subregion / Size / Address relative to `CODEHERE` / Description) with exact byte-level addressing traced from `WORD`'s and `ENVQUERY`'s own source comments rather than reconstructed from memory: WORD parsing buffer (up to 47 bytes, `CODEHERE` or `CODEHERE+3` for `S"`/`."`/`ABORT"`, up to `CODEHERE+49` worst case); HOLD buffer (34 bytes, `CODEHERE+50` to `CODEHERE+83`, i.e. `PAD-34` to `PAD-1`); PAD itself (at least 84 bytes from `CODEHERE+84` onward). Confirmed by direct arithmetic that the worst-case WORD buffer boundary (`CODEHERE+50`) exactly touches the HOLD buffer's start with no gap and no overlap, matching `WORDMAXCHARS`'s own defining formula (`PADOFFSET-HOLDMINSIZE-1-3`).

Verified visually via rendered PDF pages, not just text extraction - both the rebuilt Memory Map table and the new Transient Region table/section render cleanly across their page breaks. Confirmed the field-based Table of Contents correctly refreshed after the new content shifted page numbers (`3.

```
Glossary` moved from page 9 to 10, and every entry after it
shifted accordingly). Confirmed remaining `SIBUF`/`WORDBUF`
mentions elsewhere in the document are expected and correct -
the new section's own historical explanation, and the verbatim
Assembler Source appendix (Section 8), which legitimately still
shows this history in its own preserved inline comments - not
leftover staleness. No source-code changes were made or needed
this turn; this was a documentation-only update.
```

- **Documentation resync: Glossary and Assembler Source appendix both brought up to date with the entire MAME-testing phase of this session, after verification found both had stopped being updated right after the double-number-input work.** Confirmed the exact boundary before resyncing: `NUMBERQ` / `TRYNUM` 's appendix section was current (last thing explicitly synced), but `SIGN` , `S>D` , `LPAREN` , `ENVQUERY` , and the full `ENVTABLE` / `ENVTABLE2` expansion were all missing - the appendix still showed pre-fix code (`SIGN` / `S>D` missing their `TSTA` checks, `LPAREN` missing its stray-address fix, `ENVQUERY` missing its `PSHS X` / `PULS X` fix, `ENVTABLE` showing only the original 4 entries with `/COUNTED-STRING` still at the wrong value `31`). Replaced five appendix subsections wholesale with the current split-file source (8.1 Memory Map, 8.16 Arithmetic, 8.21 Numeric Output, 8.24 Comment Words, 8.25 Environmental Query) - same approach used successfully for the double-number sync earlier, chosen over surgical patching of individual routines within a single large paragraph.

```
Glossary: found the `ENVIRONMENT?` entry actively wrong, not
just stale - it explicitly stated "Table is incomplete: missing
/HOLD, /PAD, MAX-D, MAX-UD, WORDLISTS, FLOORED," all of which
had since been added and MAME-confirmed. Rewrote it to list
every supported query and value, and widened the stack-effect
notation to `( c-addr u -- false | i*x true )` to correctly
reflect that some queries (`MAX-D`/`MAX-UD`) return two cells,
not one. Checked `SIGN`'s, `S>D`'s, and `(`'s own glossary
descriptions before assuming they needed similar rewrites -
confirmed they didn't: each already described the *intended*
behavior the bug fixes now correctly deliver, so the bugs never
made the documentation wrong, only the implementation.

Verified: paragraph and table counts unchanged before/after
(1444 paragraphs, 3 tables); explicit presence checks for every
fix/addition across the rendered PDF; visually confirmed two
representative pages (the updated `ENVIRONMENT?` glossary entry,
and the appendix's `SIGN` routine with its full fix comment) -
both render cleanly with no formatting corruption from the
large content replacement.
```

- **`TSTDROP` added, following `TSTDUP`'s pattern exactly - second test in what's intended to become a test for every stack manipulation word.** Verifies both the stack's contents after `DROP` (the sentinel guard pushed beneath the dropped value is confirmed undisturbed and is now the new top) and the data stack pointer's movement (exactly one cell, 2 bytes, freed - `DROP`'s own net effect: `LEAU 2,U`). Traced by hand before delivery, not just structurally verified: with `TSTUB4` captured at `U_start-4` (both `TSTGUARD` and `TSTVAL1` pushed) and `TSTUAF` captured at `U_start-2` (after `DROP`'s `LEAU 2,U`), `TSTUB4-TSTUAF` computes to exactly `-2`, matching the check - the sign is deliberately opposite `TSTDUP`'s own `+2` check, since `DROP` frees a cell while `DUP` adds one. Used distinct internal labels (`DPFAIL` / `DPDONE`) rather than reusing `TSTDUP`'s (`TDFAIL` / `TDDONE`), to avoid a collision now and to establish a naming pattern (short prefix per test) that won't collide as more tests are added. Wired into `TSTSTACK` alongside the existing `JSR TSTDUP`. `UNITTESTS` reactivated (`1` -> `0`), confirmed appropriate since the user activated it independently and confirmed `TSTDUP` runs successfully on real MAME hardware - whatever earlier concern prompted excluding it was resolved by other, unrelated fixes made since. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations (checked with `UNITTESTS` both on and off, not just the file's new default); byte-exact split-file reassembly. Not yet confirmed via MAME.
- **14 more stack-manipulation tests added (`TSTSWAP`, `TSTOVER`, `TSTROT`, `TSTQDUPNZ`, `TSTQDUPZ`, `TSTDEPTH`, `TSTDDUP`, `TSTDDROP`, `TSTDSWAP`, `TSTDOVER`, `TSTNIP`, `TSTTUCK`, `TSTPICK`, `TSTROLL`, `TSTDROT`) - completing every word in the Data Stack Manipulation glossary section.** Each follows `TSTDUP` / `TSTDROP`'s established pattern exactly (guard pushed beneath, `U` snapshotted before/after, contents verified via `PULU`, pointer movement verified independently, unconditional restore at the end). `?DUP` specifically got two tests (`TSTQDUPNZ`, `TSTQDUPZ`), per the explicit request - nonzero and zero are genuinely different code paths (`QDUP` branches on the popped value), not just different inputs to the same path. Six new test-value constants added (`TSTVAL2` - `TSTVAL6`, plus `TSTGUARD` / `TSTVAL1` already existing) - all distinct from each other and from `0` / `1` / `-1`, so a test that only appears to pass due to a trivial or coincidentally-matching value would be caught. One new scratch cell (`TSTSCR`) added for `TSTDEPTH`, which independently computes its own expected value via the same `(SP0-U)/2` formula `DEPTH` itself uses, rather than assuming a fixed starting depth - robust regardless of whatever's already on the stack when it runs. `TSTPICK` / `TSTROLL` both use `u=2` as a concrete representative case (`0 PICK` is `DUP`, `1 PICK` is `OVER` - `2` is the first case distinct from both, and using the same `u` for both makes the two tests directly comparable).

```
Every test's expected stack layout was traced by hand,
```

```
instruction by instruction, against each word's real
implementation before being written - not assumed from the
glossary's stack-effect notation alone. The five trickiest
(`PICK`, `ROLL`, `2SWAP`, `2OVER`, `2ROT`) were also
cross-checked with an independent Python simulation. That
simulation caught a real bug in itself, not in the assembly
design: an incorrectly-ordered `2OVER` model that inserted both
pushed items in one step rather than modeling them as two
sequential pushes - once corrected to push explicitly in
order, it confirmed the original hand-trace was right all
along. Worth recording as a reminder that a "second check"
needs its own verification too, not blind trust.

Careful, collision-free label naming: each test uses its own
short prefix for internal `FAIL`/`DONE` labels (`SWFAIL`/
`SWDONE`, `OVFAIL`/`OVDONE`, etc.), with the double-cell variants
using their single-cell counterpart's prefix plus a `2` (e.g.
`SW2FAIL` for `2SWAP`, distinct from `SWAP`'s own `SWFAIL`) -
confirmed zero collisions via the same structural verification
used throughout this session, which would have caught any
duplicate automatically. All 16 tests (including the two
already-confirmed ones) wired into `TSTSTACK` in glossary order.
Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly. Not yet
confirmed via MAME.
```

- **`INITCODE` shifted down 3 bytes (`$FFA9` -> `$FFA6`), and the `TSTRUNNER` call site fixed to always emit exactly 3 bytes regardless of `UNITTESTS` - both per explicit request, closing a real risk: `COLDSTRT` 's own size previously depended on `UNITTESTS` (the `JSR TSTRUNNER` call existed only when `UNITTESTS=0` , emitting 0 bytes when `UNITTESTS=1`), while `INITCODE` 's position was fixed regardless - meaning toggling `UNITTESTS` could silently overflow the code into `VECTORS` without any assembler error to catch it.** Fixed by adding an `ELSE` branch: three `NOP` s (byte-for-byte the same size as the `JSR` they replace) when `UNITTESTS=1` , so the block now contributes exactly 3 bytes to `COLDSTRT` either way - its total size no longer depends on `UNITTESTS` at all. `INITCODE` shifted down by the same 3 bytes to compensate. Traced the resulting arithmetic by hand rather than assuming it worked out: the previously-confirmed 71-byte real content figure was measured under the *old* structure (0 bytes for this block, since that measurement was taken with `UNITTESTS=1`) - under the new, always-3-bytes structure, real content is reasoned to be 74 bytes (71+3), not yet re-measured by a real assembler run. The 3-byte shift in `INITCODE` 's own start and the 3-byte growth in content offset exactly, so the reasoned end (`$FFEF`) is unchanged - still one byte below

VECTORS , matching the previously-confirmed real value, if the reasoning holds; flagged as reasoned-not-measured rather than presented as confirmed, and a stale comment referencing the old $FFA9 position corrected in the same edit. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four SERIALPOLL x UNITTESTS combinations; explicitly confirmed the ELSE branch selects mutually-exclusively between the real JSR and the NOP placeholder depending on UNITTESTS , not just that the file parses; byte-exact split-file reassembly. Not yet confirmed via MAME - given this directly concerns VECTORS safety, this one specifically warrants confirming on real assembly before being considered settled, not just structural verification.

- **TSTSTACK given a header (CR , "Stack", CR), and a new subroutine TSTSARITH added - single-cell arithmetic tests covering every word in glossary section 3.4, with its own matching header ("SArithmetic"). 28 new tests total, wired into TSTRUNNER alongside TSTSTACK .** Two new negative test constants added (TSTNEG1 =-12345, TSTNEG2 =-321) - TSTVAL1 - TSTVAL6 are all positive, which wouldn't exercise the sign- dependent branches several of these words genuinely have (ABS , NEGATE , MIN / MAX , signed division, 2/ 's sign- preserving shift).

```
The two behaviors the request flagged as uncertain were both
resolved by reading this system's own glossary and source
directly, not assumed: overflow is explicitly ANS-defined as
truncation, not an error (`*`'s own glossary entry: "Signed
multiply, truncated to one cell") - tested with a case chosen
specifically to overflow 16 bits (`1000 * 1000`), verifying the
wrapped result, not an exception. Division by zero throws `-10`
- confirmed both in the glossary ("Throws -10 if n2 is zero")
and by reading `DIVCOMMON`/`STARSLASHCOMMON` directly, both of
which explicitly `THROW -10` on a zero divisor before any
division is attempted.

Divide-by-zero tests (`/`, `MOD`, `/MOD`, `*/`, `*/MOD` - one
each) use a different pattern than the rest, built around
`CATCH`: push the operands and the target word's `xt`, call
`CATCH`, verify the popped result is `-10`, and verify `CATCH`'s
own documented depth-restoration contract held (net 0 change in
`U` across the `JSR CATCH`, matching the ANS guarantee - `CATCH`
only promises stack *depth* is restored, not the `i*x` values,
confirmed precisely earlier this session). Deliberately does not
try to inspect or discard a specific count of leftover `i*x`
items - the test's own final, unconditional `LDU TSTU0` (already
established practice for every test in this framework) discards
whatever's left regardless of the count, matching the ANS
standard's own guidance to `DROP` rather than interpret them.
```

All 28 expected values computed programmatically (Python) before writing any assembly, rather than by hand, given the volume - reduces transcription-error risk across that many cases. Operand pushes and expected-value comparisons both use symbolic constant references (`#TSTVAL1` etc) wherever they correspond to an existing named constant, matching this framework's established style, rather than embedding literal hex duplicates - including, correctly, for a few cases where the *expected result* itself equals an operand's own value (e.g. `ABS` of an already-positive number returns itself unchanged, so the comparison target is `#TSTVAL1` too, not a separately-typed literal that happens to match).

Careful, collision-free label naming continued: each of the 28 new tests uses its own short prefix (`PL`, `MN`, `S1`/`S2`, `SL`/`SN`/`SZ`, `MD`/`MZ`, `SM`/`MX`, `NG`, `A1`/`A2`, `N1`/`N2`, `X1`/`X2`, `1P`/`1M`/`2P`/`2S`, `D1`/`D2`, `TS`/`TZ`, `TM`/`TX`) - confirmed zero collisions against both each other and all 17 existing stack-manipulation-test labels via the same structural verification used throughout this session.

Caught and fixed two real mistakes in my own generation process before delivery, not after: (1) a first draft of the `/` quotient-only test incorrectly used the two-result template (`/` only pushes the quotient, not a remainder - confirmed by re-reading `SLASH`'s actual code); (2) an early, mechanical symbolic-reference conversion pass left the expected-value formatting still using a helper that assumed a literal integer, which would have crashed generation the moment an expected value legitimately needed to be a symbolic reference too (the `ABS`/`MAX` cases above) - caught by actually running the generator and inspecting output, not just writing the code and assuming it worked.

Also worth recording: my own post-generation verification script produced two rounds of false "missing" alarms, both from bugs in the *checking* script itself (a text-boundary search that matched an early, incidental occurrence of a label name instead of its actual definition; and a check that only recognized `EQU`-style definitions, not label-style ones) - not from the generated source, which was correct both times. Resolved by direct inspection of the actual file content rather than trusting the automated check's first result, and by fixing the check itself before relying on it again - the same "verify your verifier" lesson already recorded elsewhere in this file.

Verified: zero duplicate symbols, dictionary chain still 224

```
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly; every symbolic
constant/label reference in the new block resolves to a real
definition somewhere in the file, checked explicitly rather than
inferred from the duplicate-symbol check alone (which doesn't
catch undefined references, only duplicate definitions - a gap
in this session's usual verification, closed for this
delivery). Not yet confirmed via MAME - given the volume (28
new tests in one delivery, several using a genuinely new pattern
involving `CATCH`), this specifically deserves a real test
before being considered settled, more than most single-word
additions this session.
```

- **RESOLVED - real assembly-blocking bug, not caught by this session's own structural verification: four internal label pairs in the arithmetic tests (`1PFAIL` / `1PDONE`, `1MFAIL` / `1MDONE`, `2PFAIL` / `2PDONE`, `2SFAIL` / `2SDONE`, for `1+`, `1-`, `2+`, `2*`) started with a digit, which the real assembler rejects (labels must start with a letter, underscore, or period) - the prefix naming scheme mechanically derived these from the Forth word names themselves, which happen to start with digits, without checking that the resulting label text was itself valid. Confirmed by the user's real assembler run, not caught here first.** Renamed to `OPFAIL` / `OPDONE` (`1+`), `OMFAIL` / `OMDONE` (`1-`), `TPFAIL` / `TPDONE` (`2+`), `TWFAIL` / `TWDONE` (`2*`) - chosen to avoid colliding with any of the other 40+ short prefixes already in use across both test files (`TS` was already taken by `STARSLASH`'s own tests, which is why `2*` couldn't reuse it despite being an obvious first choice). Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly; an explicit sweep for any other label starting with a digit anywhere in the entire file, not just the four reported - found none. Worth noting as a real gap in this session's own structural verification: the duplicate-symbol and dictionary- chain checks used throughout never actually checked label *validity* (only uniqueness), so a mechanically-generated invalid label like this could pass every check this session ran and still fail on real assembly - closed for this fix by adding the explicit digit-start sweep, worth keeping as a standing check for any future generated labels.
- **RESOLVED - two distinct real bugs in the arithmetic tests themselves, found by the user's actual MAME run (14 of 28 failed), not by this session's own verification, which had checked structural validity but never independently re-derived the expected values or checked the depth-check sign convention against each word's real arity.**

```
**Bug 1, the dominant one (13 tests): the depth-check sign was
backwards for every 2-in/1-out and 3-in/1-out test.** The
```

generator's `binop_test`/`triop_test` templates compared `TSTUB4-TSTUAF` against a *positive* `2` (or `4`), but a net-consuming operation (more cells popped internally than pushed back) *increases* `U`, which makes `TSTUB4-TSTUAF` *negative* - exactly the same sign convention already established and correctly used by `TSTDROP`/`TSTNIP`/`TSTDDROP` earlier in this same file. The template's own docstring even correctly said "-2 (one cell freed)" while the template body used `#2` - the comment and the code disagreed, and nothing checked that they matched. Affected: `TSTPLUS`, `TSTMINUS`, `TSTSTAR1`, `TSTSTAR2`, `TSTSLASH1`, `TSTSLASH2`, `TSTMODW`, `TSTMIN1`, `TSTMIN2`, `TSTMAX1`, `TSTMAX2` (all `2`->`-2`), `TSTSTSL` (`4`->`-4`). Every 1-in/1-out test (`NEGATE`, `ABS`, `1+`, `1-`, `2+`, `2*`, `2/`) and every `CATCH`-based divide-by-zero test correctly used `0` and were unaffected - confirmed by an independent, arity-derived recomputation of what every one of the 28 tests' depth check *should* be, not just the ones already reported failing.

Bug 2, a separate issue affecting `TSTSLMOD` and `TSTSTSM`: the two-result comparisons were in the wrong order. Both `SLASHMOD` and `STARSLASHMOD` push the remainder first, then the quotient last (confirmed by re-reading both routines directly: `LDD DIVREM / PSHU D` then `LDD DIVNUM`-or-`PRODLO` / `PSHU D`) - meaning the quotient is on top, popped first. The generated tests had the two expected values swapped (remainder compared first, quotient second). `TSTSTSM` had both bugs simultaneously - its depth check *and* its value order were both wrong, compounding into the same single `FAIL` report from MAME. `TSTSLMOD`'s depth check (`0`, correct for a 2-in/2-out operation) had been right all along; only the value order was wrong.

Every one of the 28 tests' depth-check sign and numeric expected values were independently re-verified against a fresh Python recomputation after the fix, not just the ones the user reported - to catch anything else wrong but not yet exercised by the specific values chosen. None found. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-exact split-file reassembly; no digit-leading labels remain (re-checked after this edit too, not just the prior one).

Root-cause note for future generated test batches: this session's structural verification (duplicate-symbol and dictionary-chain checks) cannot catch a *logically* wrong but *structurally* valid comparison value or sign - both of these

```
bugs assembled and ran without any assembler or structural-
check complaint, and would only ever have been caught by either
real execution or an independent recomputation of expected
values, which is what closed this out. Worth treating "does the
generator's own template body match its own docstring's stated
formula" as an explicit check for any future generated test
batch, not just for this one after the fact.
```

- **`TSTDARITH` added - mixed & double-precision arithmetic tests covering every word in glossary section 3.5 (14 words, 15 tests since `DABS` gets two - positive and negative cases - plus 3 divide-by-zero tests, 18 total), wired into `TSTRUNNER` alongside `TSTSTACK` / `TSTSARITH` . Header ("DArithmetic") follows the same `CR` /name/ `CR` pattern as the other two groups.** Every implementation traced by hand before any test was written, specifically to avoid repeating the previous batch's mistakes: confirmed double-cell values are pushed low- cell-first, high-cell-last (on top) consistently across every word here; confirmed which operand is `d1` vs `d2` (or dividend vs divisor) from the actual pop order, not assumed from the glossary stack notation; confirmed `FM/MOD` (floored) and `SM/REM` (symmetric) genuinely diverge on the same negative- dividend test case (`-70000 / 9320` : floored gives quotient `-8` , remainder `4560` ; symmetric gives `-7` , `-4760` - both satisfy `quot*divisor+rem=dividend` , confirming the difference is real, not an arithmetic error) - Python's native `//` / `%` used directly for the floored case, matching ANS's definition exactly, rather than hand-deriving it.

```
Depth-check values are now computed automatically from an
explicit `(cells_in, cells_out)` parameter passed to the
generator, never a separately hand-typed signed literal - a
direct structural fix for the root cause of the previous
batch's dominant bug (a docstring correctly saying "-2" while
the actual code said "2", with nothing checking they matched).
Independently re-derived what every one of the 15 core tests'
depth check *should* be from each word's real arity before
trusting the generator's own output, rather than assuming the
fix worked - all 15 matched on the first attempt.

Label collisions checked programmatically before insertion, not
assumed safe by inspection - caught and fixed 6 real collisions
this way (`DP`, `MN`, `MX`, `MS`, `DS`, `DT` all clashed with
either an existing test's prefix or, in `MSTAR`'s case, an
*internal* label already used inside the real word's own
implementation - `MSDONE` - which a manual prefix-list
cross-check would likely have missed entirely, since it's not
another test's label at all). Replaced with `PD`, `NM`, `XM`,
```

```
`MC`, `BD`, `NS` respectively, each verified against every
label in the entire file, not just other tests' prefixes.

All 15 numeric expected values independently re-verified
against fresh Python computation after generation, with proper
symbolic-constant resolution this time (the verification script
itself needed a fix mid-process - it initially flagged several
correct results as mismatches because it compared raw symbolic
references like `TSTD3HI` against literal hex without resolving
them to their real `EQU` value first; fixed by resolving symbols
before comparing, the same "verify your verifier" pattern
recorded from the section 3.4 work). All 18 tests confirmed
correctly wired into `TSTDARITH`'s call list, and every
symbolic reference confirmed to resolve to a real definition
somewhere in the file.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly; explicit sweep
confirms no digit-leading labels anywhere in the file.
**MAME-CONFIRMED**: the user reports all `TSTDARITH` tests now
pass, including `TSTMSTAR` after its own fix (logged
separately above).
```

- **RESOLVED - a real bug in `MSTAR` itself, not in `TSTMSTAR` or its push/pop order, found by the user's actual MAME run (reported as "the 2 returned values are swapped" - the real cause turned out to be a wrong computation, not a swap, though the symptom is an understandable way to describe it from the outside).** Same bug class already found and fixed multiple times this session (`PULU` doesn't set condition codes on genuine 6809), but in a new specific shape not seen before: `CLR MSIGN` runs immediately after the `PULU D` that loads `n1` , and `CLR` unconditionally sets N=0 (since it clears its destination to 0) - so the following `BPL MSN1POS` was testing `CLR` 's own result, not `n1` 's actual sign, and always branched. `n1` was never negated even when genuinely negative - its raw bit pattern got used as an unsigned magnitude instead, and `MSIGN` never got toggled for it, so the final `MNEG32` never ran either (since the code believed both operands were positive). Traced and confirmed by hand: for this test's actual inputs (`-12345 * 9320`), the buggy path computes `53191 * 9320` (treating `-12345` 's raw 16-bit pattern as the unsigned value 53191) instead of first negating to `12345` - a real, large, specific wrong answer, not a simple swap.

```
Confirmed this bug was genuinely isolated to `MSTAR`, not a
systemic pattern - both `FMSLASHMOD` and `SMSLASHREM` have the
same general shape (`CLR` calls before a sign-check branch) but
```

```
already do this correctly, with an explicit `TST PRODHI`
between their own `CLR`s and the branch, re-establishing the
correct flags - `MSTAR` was simply missing the equivalent
re-test that its siblings already had. A programmatic sweep of
the entire file for the same unguarded shape (`PULU D` followed
within a few lines by a `CLR` then a signed branch, with no
intervening flag-setting instruction) found no other instances.

Fixed with `TSTA` (testing D's high byte, the same fix pattern
already used for `TRYNUM`/`STOD`/`SIGN` earlier this session)
inserted between `CLR MSIGN` and `BPL MSN1POS`. Verified the fix
by hand-tracing the corrected path for this test's exact
inputs: `n1` now correctly detected as negative, correctly
negated to `12345` before the unsigned multiply, `MSIGN`
correctly toggled, final `MNEG32` correctly applied - produces
exactly `TSTMSTAR`'s original expected values (`$F924`, `$64D8`)
with zero changes needed to the test itself. Verified: zero
duplicate symbols, dictionary chain still 224 entries intact
across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-
exact split-file reassembly. **MAME-CONFIRMED**: the user
reports all `TSTDARITH` tests now pass, including `TSTMSTAR` -
the fix holds on real hardware, not just by hand-trace.
```

- **RESOLVED - `TSTSELECTOR` transferred in from the user's own tested approach, addressing a real problem this session hadn't caught: assembling all three test groups (`TSTSTACK` , `TSTSARITH` , `TSTDARITH`) together exhausts the available unused ROM space.** New flag, `TSTSELECTOR EQU 2` , gates each group's own call-list *and* its own test-body definitions separately (two `IFEQ TSTSELECTOR-N` / `ENDC` pairs per group, `N` = 0/1/2 for `TSTSTACK` / `TSTSARITH` / `TSTDARITH` respectively) - so only the currently-selected group's test code actually compiles into the ROM image, not all three at once. Each group's header message (`CR` /name/ `CR`) stays unconditional, so it prints regardless of which group is selected; only the actual test-invoking calls and their bodies are gated.

```
Received as an uploaded file rather than a description, so
verified by diffing the user's file against this session's
version rather than assuming the transfer was needed at all -
confirmed the diff was genuinely just this mechanism (plus this
session's own `MSTAR` bug fix, absent from the user's copy
since it predates that fix) before applying anything. Applied
by locating each of the six exact insertion points in this
session's own file and inserting the identical text, then
diffed the result against the user's file again to confirm an
exact match (aside from trailing-whitespace-only differences on
```

```
a few blank lines, and the `MSTAR` fix, correctly present only
in this session's copy).

Verified beyond the standard structural check: extended the
duplicate-symbol/dictionary-chain verification to cover
`TSTSELECTOR`'s three values crossed with both `SERIALPOLL`
settings (8 relevant combinations, since `TSTSELECTOR` only
matters when `UNITTESTS=0`) - zero duplicate symbols, dictionary
chain intact (224 entries) in all eight. Also explicitly
confirmed the mechanism achieves its actual goal, not just that
it assembles cleanly: simulated each `TSTSELECTOR` value and
confirmed only that one group's test bodies are present in the
output (checked via a representative test name per group), with
the other two groups' bodies genuinely absent, not just
unreachable - and confirmed total compiled size per selector
value is roughly a third of what all three groups combined
would be. Byte-exact split-file reassembly confirmed.
**MAME-CONFIRMED**: the user reports the whole `TSTDARITH`
group (`TSTSELECTOR=2`) now assembles and runs successfully,
validating the mechanism end to end, not just structurally.
```

- **RESOLVED - `UNITTESTS` and `TSTSELECTOR` converted from plain `EQU` to an `IFNDEF` / `SET` / `ENDC` fallback-default pattern, per explicit request, supporting future override via lwasm's `-D` command-line option without editing the source.** `-D` pre-defines a symbol before the source is processed, so `IFNDEF` correctly detects and skips the fallback `SET` when a value was supplied that way, leaving the `-D` -provided value in place; when no `-D` was given, the symbol is genuinely undefined at that point, `IFNDEF` fires, and the fallback value applies. `TSTSELECTOR` 's fallback preserves its current working value (2) - the request's own example showed 0, but that was illustrating the pattern, not asking to reset the actual selector.

```
**`UNITTESTS`'s flag meaning deliberately reversed at the same
time, per explicit request**: both `IFEQ UNITTESTS` occurrences
(the main test-framework body, and the `COLDSTRT` call site)
changed to `IFNE UNITTESTS` - inverting the semantics from
"0=included" to "0=excluded, nonzero=included". This makes 0
(or no `-D` at all) the safe, production default - test-
framework code and its `TSTRUNNER` call site both genuinely
absent from a default build - with testing now opt-in via
`-DUNITTESTS=1` rather than opt-out via editing a plain `EQU`.
Both `IFEQ`→`IFNE` changes applied together deliberately, not
independently - leaving one as `IFEQ` while the other became
`IFNE` would have left the test-framework body and its own call
site testing opposite conditions, a real risk of building
correctly but crashing at the call site (or vice versa) that
```

```
was checked for explicitly, not just assumed avoided by intent.
Updated stale comments referencing the old "0=included,1=
excluded" convention in both places, including the historical
note on the `COLDSTRT` call site's own `NOP`-padding fix from
earlier in the session, which had specifically referenced
`UNITTESTS=1` as the excluded case - no longer accurate under
the reversed meaning.

This session's structural verification simulator extended to
handle the three new constructs (`IFNDEF`, `SET`, `IFNE`), none
previously present in this file - `IFNDEF` modeled via an
explicit set of "defined" flags (mirroring what `-D` would
predefine), `SET` treated identically to `EQU` for verification
purposes, `IFNE` as the logical inverse of the existing `IFEQ`
handling. Verified across scenarios a plain duplicate-symbol
check wouldn't distinguish: (1) no `-D` at all - confirmed the
entire test framework, including `TSTRUNNER` and every
individual test body, is genuinely absent, not just
unreachable, and the call site correctly falls through to its
`NOP` placeholder; (2) `-DUNITTESTS=1` with each `TSTSELECTOR`
value explicitly passed - confirmed correct group selection
still works under the reversed `UNITTESTS` convention; (3)
`-DUNITTESTS=1` alone, `TSTSELECTOR` *not* separately overridden
- confirmed it correctly falls back to its own default (2)
rather than picking up `UNITTESTS`' value or failing to resolve.
All scenarios: zero duplicate symbols, dictionary chain intact
(224 entries). Byte-exact split-file reassembly confirmed. Not
yet confirmed via real lwasm - simulated `-D` behavior here,
not run against the actual toolchain's own `-D` implementation,
which is worth checking given this is new assembler-feature
territory for this file, not just new source structure.
```

- **RESOLVED - unit test framework block comment fixed, per explicit request: "UNITTESTS=1 removes it entirely" was stale after the flag-meaning reversal, and would have told a reader the exact opposite of the current behavior.** Rewrote to describe the current (0/undefined=excluded, nonzero=included) convention, and added the requested example lwasm command line (`--define=UNITTESTS --define=TSTSELECTOR=2`) at the end of the block comment. Also proactively fixed three further references to the old convention in `INITCODE` 's own historical comment (describing an earlier fix made before the reversal existed) - left unfixed, these would have recreated the same confusion just flagged, one layer further into the file.
- **`TSTLOGIC` added - logic, shift, and address-arithmetic tests (glossary section 3.6, 12 words, 12 tests), wired into `TSTRUNNER` and gated as `TSTSELECTOR` 's fourth group (`-3`).** Several of these words modify the top of stack in place (`LDD` / op/ `STD` , never

`PULU` / `PSHU` at all - `INVERT`, `CELLS`, `CELL+`, `CHARS`, `CHAR+`, `ALIGNED`) - tests verify what's observable via the stack regardless of implementation mechanism, reusing the same generic push/pop test template as every previous group without needing special-casing. Three words (`CHARS`, `ALIGN`, `ALIGNED`) are documented no-ops on this system; `CHARS` / `ALIGNED` got ordinary single-value identity tests, while `ALIGN` - which takes no stack arguments at all - got a dedicated test pushing two decoy values to confirm the *whole* stack is undisturbed, not just a single value's persistence (the same template handles this too, called with `cells_in=0, cells_out=0` and no real "operand" semantics). `RSHIFT` tested against a negative input specifically, since it's documented logical (zero-fill) rather than arithmetic (sign-preserving) - the same reasoning already applied to `2/` and `FM/MOD` -vs- `SM/REM` in earlier groups, confirmed to give a genuinely different result than an arithmetic shift would.

```
Label collisions checked programmatically before insertion -
caught 3 real ones this way (`LS`, `RS`, `HS` all collided with
internal labels already inside `LSHIFT`'s, `RSHIFT`'s, and
`HOLDS`'s own implementations - `LSDONE`/`RSDONE`/`HSDONE` -
the same class of issue as `MSTAR`'s own `MSDONE` collision
found earlier). Replaced with `L2`, `R2`, `C3`, each re-verified
against every label in the file.

**A real mistake caught and fixed before delivery, not after**:
the initial insertion left `TSTSELECTOR`'s second `IFEQ`/`ENDC`
pair (wrapping `TSTLOGIC`'s own test-body definitions) with no
closing `ENDC` at all - the wrapper and generated test bodies
were concatenated without one, unlike the `TSTDARITH` transfer
earlier, which needed and got an explicit closing `ENDC` inserted
by hand. The open block fell through and silently consumed the
pre-existing outer `ENDC` that closes the entire `UNITTESTS`
block, leaving everything unbalanced by one. This didn't fail
the routine duplicate-symbol check (which doesn't verify
`IFEQ`/`ENDC` balance at all) - it was caught by extending
verification to actually simulate the production-default build
(no `-D` at all) and finding the dictionary chain walk returned
*zero* entries instead of 224, a stark, unmissable signal once
checked, but one that a narrower check would have missed
entirely. Root cause confirmed precisely (an explicit open/close
trace of every `IFEQ`/`IFNE`/`IFNDEF`/`ENDC` in the file,
showing depth ending at 1 instead of 0) before fixing, not
guessed at. Fixed by adding the missing `ENDC`; `IFEQ`/`IFNE`/
`IFNDEF` vs `ENDC` counts now balanced (15/15) file-wide.

Verified after the fix: every one of the 12 depth-check values
independently re-derived from real arity and confirmed correct;
```

```
every numeric expected value independently re-verified against
fresh computation with proper symbolic-constant resolution;
every symbolic reference confirmed to resolve to a real
definition; all 12 tests confirmed wired into `TSTLOGIC`. Full
scenario matrix re-run after the `ENDC` fix specifically (not
just re-trusted): production default (no `-D`) now correctly
shows the full 224-entry chain again, `-DUNITTESTS=1` crossed
with all four `TSTSELECTOR` values and both `SERIALPOLL`
settings (10 scenarios total) all pass; explicitly confirmed
`TSTSELECTOR=3` includes only `TSTLOGIC`'s own bodies, with the
other three groups' bodies genuinely absent. Byte-exact split-
file reassembly confirmed. **MAME-CONFIRMED**: the user reports
all `TSTLOGIC` tests pass, including `ALIGN`'s dedicated no-op
test and `RSHIFT`'s negative-input logical-shift case - the
`ENDC` fix holds on real hardware too, not just in simulation.
```

- **`TSTCOMPARE` added - comparison tests (glossary section 3.7, 14 words, 15 tests since `WITHIN` gets two cases), wired into `TSTRUNNER` and gated as `TSTSELECTOR`'s fifth group (`-4`).** Signed-vs-unsigned pairs (`</U<` , `>/U>`) each tested with an operand pair that would give the *opposite* answer under the other convention (`TSTNEG1` 's raw bit pattern is a large unsigned magnitude despite being a negative signed value) - confirms genuine sign-awareness rather than an accidentally- shared implementation. `WITHIN` tested against a wraparound range specifically (`n2` near `$FFFF` , `n3` wrapped past `$0000`), both inside and outside cases - the documented special case ("handles wraparound ranges correctly," via unsigned-offset comparison) its own implementation exists to handle, not just an ordinary non-wrapping range; traced the real implementation by hand first to confirm it computes `(n1-n2) < (n3-n2)` unsigned before designing the test, not assumed from the glossary's one-line description. `D<` / `DU<` both tested with equal high cells and different low cells - the documented tie-break case ("low cells unsigned only if [high cells] equal") a naive high-cell-only comparison would get wrong.

```
Label collisions checked programmatically before insertion -
caught one real one this way (`DU`, intended for `DULESSW`,
collided with `DUDONE` already inside `DUMP`'s own
implementation, part of the unrelated Tools word set - the same
class of issue as `MSTAR`/`LSHIFT`/`RSHIFT`/`HOLDS` found in
earlier groups). Replaced with `DZ`, re-verified against every
label in the file.

**Applied the lesson from the previous group's real mistake
immediately this time**: checked the file's `IFEQ`/`IFNE`/
`IFNDEF` vs `ENDC` balance right after insertion, before any
other verification - confirmed balanced (17/17) on the first
```

```
check, meaning both `TSTSELECTOR-4` wrap points (call-list and
test-body definitions) got their closing `ENDC` correctly this
time, not caught after the fact via the production-build
simulation like last time.

Verified: every one of the 15 depth-check values independently
re-derived from real arity and confirmed correct; every numeric
expected value independently re-verified against fresh
computation with proper symbolic-constant resolution; every
symbolic reference confirmed to resolve to a real definition;
all 15 tests confirmed wired into `TSTCOMPARE`. Full scenario
matrix re-run with `TSTSELECTOR`'s fifth value included (12
scenarios: production default x2, `-DUNITTESTS=1` crossed with
all five `TSTSELECTOR` values x2 `SERIALPOLL` settings) - all
pass; explicitly confirmed `TSTSELECTOR=4` includes only
`TSTCOMPARE`'s own bodies, with the other four groups' bodies
genuinely absent. Byte-exact split-file reassembly confirmed.
**MAME-CONFIRMED**: the user reports all `TSTCOMPARE` tests
pass, including the `WITHIN` wraparound cases and the `D</DU<`
tie-break cases - the `IFEQ`/`ENDC` balance fix applied at
insertion time (learned from the previous group's mistake) held
on real hardware, not just in simulation.
```

- **`TSTCTRLFLOW` added - control-flow tests (glossary section 3.8, 22 words, 20 tests), wired into `TSTRUNNER` and gated as `TSTSELECTOR`'s sixth group (`-5`). A fundamentally different testing problem from every prior group: these are compile-time, immediate, code-generating words, not runtime operations on operands.** Every implementation traced by hand first (`IF` / `THEN` / `ELSE` , `BEGIN` / `UNTIL` / `AGAIN` / `WHILE` / `REPEAT` , `RECURSE` , `DO` / `?DO` / `LOOP` / `+LOOP` , `I` / `J` / `LEAVE` / `UNLOOP` , `EXIT` , `CASE` / `OF` / `ENDOF` / `ENDCASE` , plus `PATCH` / `CCALL` / `CODECOMMA` / `ZBRANCH` / `BRANCH` and the return-stack loop-frame layout) - confirmed the "control-flow stack" is just the ordinary data stack (`IF` pushes `(patch-addr, TAGFWD)` ; `THEN` pops and patches), and everything reads/writes through `CODEHERE` , a plain variable, not tied to a real dictionary entry.

```
**Test methodology**: each test redirects `CODEHERE` to a new
scratch buffer (`TSTCBUF`, 80 bytes), calls the real compile-
time words directly (`JSR IF`, `JSR THEN`, `JSR DO`, etc - the
actual routines the compiler itself calls, not a hand-simulated
imitation), restores `CODEHERE`, then executes the compiled
snippet directly. Tests the real interaction between compile-
time correctness (right bytes, right patched offsets) and
runtime correctness (right control flow) in one coherent check,
without needing the outer interpreter, `FIND`, or a dictionary
```

entry. Given the planned future ANS test suite for broader standards-compliance coverage, this deliberately focuses on the compile/patch mechanism itself, not full end-to-end parsing. `UNLOOP` is the one exception - a genuine runtime no-op (bare `RTS`), tested directly like `ALIGN`'s own test, needing none of this harness. `RECURSE` needed its own scratch (`TSTFHDR`, a fake dictionary header with a known CFA, `LATEST` temporarily redirected to it) since it reads `LATEST` directly. `EXIT` needed the real `CSP` variable set correctly (matching what `:` does at the start of a real definition), since its own compile-time frame-counting scan depends on it.

Every compiled snippet's exact byte layout and patched offset values were hand-traced before being trusted - not just "it assembled." Two genuinely difficult cases specifically: `IF`/`ELSE`/`THEN` (verified IF's patched offset lands exactly at the ELSE-body's start, ELSE's own patched offset lands exactly past it) and `BEGIN`/`WHILE`/`REPEAT` (verified WHILE's forward offset lands exactly at the final `RTS`, REPEAT's back-edge lands exactly at `BEGIN`).

A real, substantive finding, not just a test-writing detail: traced `DOTEST`'s actual termination logic (simulated the real increment-and-compare-equal check) and confirmed `DO` with `limit=index` takes a full 65536-iteration wraparound to naturally reconverge on equality - true to the letter of the glossary's "runs at least once" but not in any fast sense. Deliberately left untested rather than build something impractical for a boot-time check or guess at it. `?DO`'s own equivalent case (`TSTQDOLPEQ`) IS tested - confirmed by the same kind of trace that its skip check happens before the loop is entered at all, fast and safe.

Distinguished `LEAVE` from `EXIT` precisely by hand-tracing when each actually takes effect: `LEAVE` only sets a flag, checked at the *next* `LOOP`, so the current iteration's trailing code still runs (`TSTLEAVE`'s sum is 0+1+2+3=6, including the iteration where `I`=3 triggers `LEAVE`); `EXIT` fires immediately where encountered, so the same-shaped test (`TSTEXIT`) gives 0+1+2=3, not including `I`=3's own iteration. Getting this backwards would have been a real, silent bug in the tests themselves, not caught by any structural check.

Error-path tests (`TSTTHENZ`/`TSTUNTILZ`/`TSTENDOFZ`, covering "throws -22 on tag mismatch") reuse the established `CATCH` pattern from earlier sections directly - confirmed by reading `CFERR` that the error path never touches `CODEHERE` at all, so

no redirect is needed for these three specifically, unlike every other test in this group.

Two real mistakes caught and fixed during this batch, not after: (1) three depth-check sign errors (`TSTIFT2`, `TSTRECUR`, `TSTDOLP`) - the same class of mistake as earlier sections - caught by systematically re-deriving every depth value from actual cells-in/cells-out arithmetic rather than trusting the first pass, before any insertion happened. (2) a genuinely new mistake: concatenating the 14 draft files via a shell glob (`tstctrl_part*.txt`) sorted lexicographically, not numerically, scrambling the test bodies' order in the first insertion attempt (`part1` followed by `part10`-`part14`, then `part2`-`part9`) - likely harmless for actual assembly (forward references are fine), but sloppy and not what was intended. Caught via a reference-verification check that found expected words apparently missing from the inserted block, traced precisely to the real cause (wrong extraction boundary revealing the reordering, not missing content) rather than assumed fixed, and the whole insertion redone with correct numeric ordering before delivering anything.

Label collisions checked programmatically before insertion - caught 6 real ones this way, all simple prefix reuse from this session's own earlier sections (`DZ`, `UL`, `PL`, `DL`, `TZ`, `UZ`), replaced with `EO`, `UO`, `PO`, `DW`, `T3`, `U3`, each re-verified against every label in the file.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all 14 relevant `SERIALPOLL`x `UNITTESTS`x`TSTSELECTOR` scenario combinations (production default, and `-DUNITTESTS=1` crossed with all six `TSTSELECTOR` values and both `SERIALPOLL` settings); every one of the 20 tests' depth-check values independently re-derived from real arity and confirmed correct; every symbolic and word reference confirmed to resolve to a real definition (a small number of apparent false positives - bare `F`/`O` tokens - traced to the quoted character literals in `TSTRECUR`'s fake header construction, not real gaps); all 20 tests confirmed wired into `TSTCTRLFLOW`; explicitly confirmed `TSTSELECTOR=5` includes only this group's own bodies. Byte-exact split-file reassembly confirmed. **MAME-CONFIRMED**: the user reports all `TSTCTRLFLOW` tests pass - including the `IF`/`ELSE`/`THEN` dual-branch cases, `BEGIN`/`WHILE`/`REPEAT`'s two-offset patch, `RECURSE`'s fake-header mechanism, every `DO`-family loop variant, the `LEAVE`-vs-`EXIT` timing distinction, `CASE`'s match/no-match paths, and all three tag-mismatch error paths.

```
This was this session's most structurally complex test group -
the redirected-`CODEHERE`, compile-then-execute harness holds
up on real hardware, not just in simulation, validating the
whole methodology for any future control-flow-adjacent work.
```

- **TSTDEFWORDS added - defining-words tests (glossary section 3.9, 17 words, 12 tests since VALUE / TO and IS / ACTION-OF are each combined into one test), wired into TSTRUNNER and gated as TSTSELECTOR 's seventh group (-6). A harder testing problem than section 3.8: these words don't just compile code, they parse a name from the input source and build real dictionary headers.** Every implementation traced by hand first (HEADER , shared by : / CREATE ; the CREATE / DOES> trampoline mechanism via DODOES / SETDOES ; VARIABLE / CONSTANT / 2VARIABLE / 2CONSTANT / BUFFER: / VALUE / TO / DEFER / DEFER@ / DEFER! / IS / ACTION-OF / MARKER) - confirmed HEADER calls WORD directly to parse the new word's name, something nothing in section 3.8 needed.

```
**Harness extended beyond section 3.8's `CODEHERE`-only
redirect**: every test now also redirects `DPHERE` (a new 40-
byte scratch dictionary buffer, `TSTDBUF`), `VARHERE` (a new
20-byte scratch buffer, `TSTVBUF`), and `SRCADDR`/`SRCLEN`/
`TOIN` (a fake name, "TESTWD", reused safely across every test
here since each redirect/restore cycle is fully isolated) - and
established a new convention: save `CODEHERE` immediately before
calling the defining word (this becomes the newly-defined
word's own CFA), so its compiled trampoline can be executed
afterward to verify runtime behavior.

Worked out `SETDOES`'s "double return" mechanism precisely by
hand before trusting it: confirmed a direct call into the
compiled "JSR SETDOES" instruction itself (rather than building
an intermediate wrapper word) naturally supplies exactly the
two return-stack levels it expects - its own `JSR` call provides
one, this test's own call provides the other - without extra
scaffolding. Also traced why bare `CREATE` (never followed by
`DOES>`) isn't tested standalone: its placeholder behavior field
is a genuine compiled `JSR DOESRT0` instruction, while every
other defining word compiles a raw address there instead (via
`CODECOMMA`, not `CCALL`) - `DODOES` reads that field as data
either way, which only produces a valid jump target for the
raw-address form.

**A real, structurally significant bug found and fixed during
this batch, not after**: `HEADER` writes to the real `LATEST`
variable unconditionally - unlike `CODEHERE`/`DPHERE`/`VARHERE`,
```

there's no redirect for it. The first 9 tests written (`TSTVAR` through `TSTDEFER2`) didn't save/restore it, which would have left the real dictionary's `LATEST` pointer corrupted, pointing at scratch memory, after every one of them ran - a genuine risk to the real dictionary, not just a test-isolation nicety. Caught by tracing `HEADER`'s own code directly rather than assuming symmetry with the other three pointers, and fixed across all 8 affected files before any insertion happened, re-verified afterward by direct inspection of each save/restore block's exact placement, not just re-trusted.

`MARKER`'s own test needed the opposite execution order from every other test in this group: `DOMARKER` (confirmed by reading it directly) also writes to the real `DPHERE`/`CODEHERE`/`VARHERE`/`LATEST` unconditionally, so that test executes the marker word *while* still redirected, restoring the real environment only afterward - reversing this order would have corrupted the real dictionary pointers with scratch addresses.

Label collisions checked programmatically before insertion - zero found this time (all chosen prefixes verified safe on the first attempt). Learned the lesson from section 3.8's ordering mistake and applied it from the start this time: used numeric (not lexicographic) file ordering throughout, and checked the `IFEQ`/`ENDC` balance immediately after insertion rather than discovering a problem later - both came back clean on the first attempt.

Verified: every one of the 12 depth-check values independently re-derived from real arity and confirmed correct (one sign error caught and fixed during writing, before insertion); every symbolic and word reference confirmed to resolve to a real definition (a handful of apparent false positives - bare character-literal tokens from the repeated "TESTWD" name construction - traced to their real cause, not just dismissed); all 12 tests confirmed wired into `TSTDEFWORDS`. Full scenario matrix re-run with `TSTSELECTOR`'s seventh value included (16 scenarios total) - all pass; explicitly confirmed `TSTSELECTOR=6` includes only this group's own bodies. Byte-exact split-file reassembly confirmed. **MAME-CONFIRMED**: the user reports all `TSTDEFWORDS` tests now pass, closing out three real, distinct issues found via real hardware over several rounds - a byte-overflow needing `LBNE` (a genuine `BNE`/`ISFAIL` short-branch overflow, unrelated to this section's own logic), a `WORD`-overwrites-`CODEHERE` corruption bug in `TSTVALTO`/`TSTISOF`'s second phases, a `SETDOES`/`LATEST`-ordering bug in `TSTCRDOES`,

```
and a wrong comparison target in `TSTMARKER`'s own verification
logic (not a bug in `MARKER` itself). This section's own
complexity - name-parsing, real header-building, the trampoline
mechanism - proved out exactly the concern flagged when this
group was first delivered.
```

- **RESOLVED - real assembly-blocking bug found by the user's actual lwasm run, not caught by this session's own verification: four `BNE` instructions in `TSTVALTO` and `TSTISOF` exceeded short-branch range (`Byte overflow`).** Same bug class as `DULINE`'s own already-documented fix elsewhere in this file - `TSTVALTO` and `TSTISOF` are this section's two combined, two-round tests (`VALUE` + `TO`, `IS` + `ACTION-OF`), and their earliest checks sit far enough before the shared `FAIL` label - past the entire second round's own code - to exceed a short branch's range. Fixed the exact four flagged lines (verified by line number, not by text pattern, since identical `BNE VTFAIL` / `BNE ISFAIL` text appears multiple times in each test) by converting them to `LBNE` - confirmed this is an established, already-proven mnemonic in this file rather than assumed. Trusted the assembler's own precise report rather than pre-emptively converting every `BNE VTFAIL` / `BNE ISFAIL` occurrence - 7 of the 11 total instances were not flagged and were left as-is, matching their genuinely shorter distance to the target. Confirmed by reasoning through it that the fix doesn't newly endanger those remaining short branches either: the added bytes from converting `BNE` to `LBNE` sit entirely before every one of them, which doesn't change their own distance to the shared target at all.

```
Verified: `IFEQ`/`ENDC` balance unaffected (still 21/21,
re-checked after this edit rather than assumed unaffected by a
pure branch-instruction change); zero duplicate symbols,
dictionary chain still 224 entries intact across all 16
relevant `SERIALPOLL`x`UNITTESTS`x`TSTSELECTOR` scenario
combinations; exact counts of `LBNE`/remaining-`BNE` confirmed
(4 and 7 respectively, matching the fix precisely). Byte-exact
split-file reassembly confirmed. This specific fix awaits its
own confirmation via a real lwasm run - the user's own report is
what caught it, this response addresses exactly what was
reported.
```

- **RESOLVED - real crash found by the user's actual MAME run, not caught by this session's own verification: `TSTVALTO` jumped into invalid memory at `TSTCBUF`'s own address on its second `JSR ,X`.** Root cause confirmed by reading `WORD`'s own code directly rather than guessed at: `WORD` writes its parsed- token output directly at `CODEHERE`, by design ("matches the ANS transient-region contract, where WORD's region is expected to be overwritten by whatever gets compiled/allocated next" - `CODEHERE` itself isn't advanced by this). `TSTVALTO` redirects `CODEHERE` to `TSTCBUF` a

second time for its own `TO` phase's name-parse, after `TSTCBUF` already held `VALUE` 's own compiled trampoline from the first phase - `WORD` 's own internal parse (called by `TOW`) silently overwrote it. Confirmed precisely against the user's own memory dump: a length byte (`06`) followed by "TESTWD" sitting exactly at `TSTCBUF` 's address, where the trampoline's real `JSR DODOES` opcode should have been - not a vague "probably right," the exact byte pattern `WORD` 's own documented behavior predicts.

```
Checked every other test in this section for the same pattern
before assuming this was isolated - confirmed it's structurally
possible only for this section's two "combined, two-phase"
tests (`TSTVALTO`, `TSTISOF`), since every other test does a
single name-parse with nothing to conflict with. Found
`TSTISOF` had a related but more serious version of the same
bug: its `IS` and `ACTION-OF` phases had *no* `CODEHERE`
redirect at all (not even the wrong one) - meaning `WORD`'s own
internal parse there would have written into the real,
unredirected system `CODEHERE`, unsafe during boot-time testing
before `COLD` has set it to anything meaningful. The `IS` phase
specifically mattered more than `ACTION-OF`'s, since it runs
*before* this test's own execution step - the same class of
corruption that crashed `TSTVALTO`, just not yet triggered by
MAME's own test run order.

Fixed with a new, dedicated scratch buffer (`TSTCBUF2`, 20
bytes) used for every "second phase" name-parse in both tests,
keeping it structurally distinct from `TSTCBUF` so a later
parse can never overwrite an earlier phase's still-needed
compiled trampoline. Confirmed `TOW`/`ISW`/`ACTIONOF`'s own
interpreting-mode paths don't touch `DPHERE`/`VARHERE` (traced
directly - they only call `WORD`+`FIND`+`TOBODY` plus a direct
memory store, none of which touches those two), so the fix only
needed to cover `CODEHERE`.

Verified: `IFEQ`/`ENDC` balance unaffected (21/21, re-checked
rather than assumed); both tests' own depth-check values
re-confirmed unchanged by this purely corrective edit (`[2,2]`
for `TSTVALTO`, `[2]` for `TSTISOF`, matching their state before
this fix); `TSTCBUF2` confirmed correctly declared and
referenced exactly where intended; zero duplicate symbols,
dictionary chain still 224 entries intact across all 16 relevant
`SERIALPOLL`x`UNITTESTS`x`TSTSELECTOR` scenario combinations.
Byte-exact split-file reassembly confirmed. This specific fix
awaits its own confirmation via MAME - the user's own crash
report is what caught the root cause, this response addresses
exactly what was found, including the related `TSTISOF` issue
```

```
the crash report itself didn't directly point to.
```

- **RESOLVED - real bug found by the user's actual MAME run, pinpointed precisely from a register/memory dump, not guessed at: `TSTCRDOES` failed because `SETDOES` patched the wrong word entirely.** Root cause: this test's own restore block set `LATEST` back to its real, original value *before* triggering `SETDOES` (via a direct call into the compiled "JSR SETDOES" instruction) - but `SETDOES` (confirmed by re-reading its own code) reads `LATEST` directly to find which header's behavior field to patch. With `LATEST` already restored, `SETDOES` patched whatever real word `LATEST` actually pointed to, not `TESTWD` - leaving `TESTWD` stuck on its original `DOESRT0` placeholder (push PFA, return - a plain `VARIABLE` -like behavior), never running `@ 1+` at all.

  ```
  Confirmed precisely against the user's own register/memory
  dump before fixing anything: the crashing `CMPD #6` compared
  against `$022C`, exactly matching `TESTWD`'s own computed PFA
  address (`TSTCBUF+7`, the self-referential-PFA pattern already
  documented elsewhere in this file) - the exact value `DODOES`
  pushes under `DOESRT0`'s default behavior, not a coincidence.

  Fixed by reordering: `LATEST` now stays pointed at `TESTWD`
  until *after* the `SETDOES` trigger completes, only restored to
  its real value afterward - `CODEHERE`/`DPHERE`/`VARHERE`/
  `SRCADDR`/`SRCLEN`/`TOIN` can still restore early, since
  `SETDOES` doesn't depend on any of those (confirmed by reading
  its code - it only touches `LATEST` and the computed behavior-
  field address).

  Verified: `IFEQ`/`ENDC` balance unaffected (21/21); this test's
  own depth-check value unchanged (`2`); zero duplicate symbols,
  dictionary chain still 224 entries intact across all 16
  relevant scenario combinations. Byte-exact split-file
  reassembly confirmed. This specific fix awaits its own MAME
  confirmation.
  ```

- **RESOLVED - `TST2VAR` confirmed passing after the `TSTCRDOES` fix above (the user's own re-run confirmed it), without any change of its own needed.**
- **RESOLVED - `TSTMARKER` 's failure, precisely diagnosed from the user's own register/memory dump, turned out to be a bug in this test's own verification logic, not in `MARKER` / `DOMARKER` themselves - the real dictionary/compile mechanism was already correct.** The failing comparison was `TSTCSAV2` (CODEHERE right after executing the marker) against `TSTMKCOD` (a snapshot of CODEHERE captured right *after* `MARKERW` finished building its own header) - decoded precisely against the user's own

memory dump: `TSTCSAV2` held `$0225` (`TSTCBUF` exactly), `TSTMKCOD` held `$0234` (`TSTCBUF` +15, the full compiled size of `MARKERW`'s own trampoline+snapshot structure - confirmed byte for byte by re-reading `MARKERW`'s complete implementation, not estimated).

```
Traced why these genuinely differ rather than assuming a
simple off-by-some-amount error: `MARKERW`'s own internal
snapshot (`MKDP`/`MKCODE`/`MKVAR`/`MKLATEST`) is taken at its
very first four instructions, before `HEADER` or any of its own
compiling runs at all - meaning `DOMARKER` correctly restores
to the state *before* the marker word itself was created, not
the state right after. This is exactly `MARKER`'s own documented
behavior ("forgets itself too", not just what comes after it) -
this test's own comparison target (`TSTMKCOD`, captured too
late) was simply wrong from the start, not the mechanism it was
testing.

Fixed by comparing directly against `TSTCBUF`/`TSTDBUF`/
`TSTVBUF` (the real, known pre-creation values) instead, and
removed the now-unnecessary `TSTMKCOD`/`TSTMKDP`/`TSTMKVAR`
capture step and their now-unused scratch declarations entirely,
rather than leaving dead, misleading allocations behind. Updated
this test's own header comment, which had described the old
(wrong) design, to accurately reflect the fix and why it's
correct.

Verified: `IFEQ`/`ENDC` balance unaffected (21/21); this test's
own depth-check value unchanged (`0`); confirmed no remaining
code references to the removed constants (only an explanatory
comment, which is harmless); zero duplicate symbols, dictionary
chain still 224 entries intact across all 16 relevant scenario
combinations. Byte-exact split-file reassembly confirmed. This
specific fix awaits its own MAME confirmation.
```

- ** `TSTRETSTACK` added - return-stack tests (glossary section 3.3, 6 words, 4 tests since `>R` / `R>` and `2>R` / `2R>` are each combined into one round-trip test), wired into `TSTRUNNER` and inserted in the source **in glossary order** (right after `TSTSTACK`'s own tests, before `TSTSARITH`'s), per explicit request - **not** at the end of the file like every prior group. Gated as `TSTSELECTOR`'s eighth value (`-7`), the next available number rather than a value matching this glossary-order position - per explicit request, `TSTSELECTOR`'s own numbering stays as-is for now, with a renumbering-to-match-glossary-order pass explicitly deferred to a future request, not attempted here.

```
A genuinely different testing problem from every prior group:
```

these words operate directly on the return stack (`S`) - the same stack holding real subroutine return addresses, including this test framework's own. Traced each word's `PULS`/`PSHS` juggling by hand first: `>R`/`R>`/`2>R`/`2R>` all temporarily lift the caller's own return address off `S`, do the actual move, then restore it on top - so a moved value ends up nested one level inside the *current* subroutine's own return-stack frame, safely retrievable by a later `R>`/`2R>` in the same body. Every `>R`/`2>R` in these tests is balanced by a matching `R>`/`2R>` before this group's own `RTS` - leaving one unbalanced would corrupt the path back to whatever called this test, a real risk specific to this section that no prior group had to guard against.

`TSTTOR`/`TSTTWOTOR` each include an intermediate depth check (right after the move-out, before the move-back) specifically because a pure round-trip check could pass even if both halves were broken no-ops - the value would simply never have left. `TSTRFETCH`/`TSTTWORFETCH` verify "non-destructive" for real: peek, then retrieve the original afterward and confirm it's still correct, not just that the peek itself returned the right value once.

Label collisions checked programmatically before insertion - caught one real one this way (`TW`, colliding with `TSTBWR`'s own prefix from section 3.6's `BEGIN`/`WHILE`/`REPEAT` test), replaced with `T2F`, re-verified against every label in the file. Caught and fixed the same `IFEQ`/`ENDC` mistake as section 3.8's own first attempt - forgot the second wrap's closing `ENDC` around the test bodies - but this time caught it immediately via the balance check run right after insertion, not discovered later via a broken production-build simulation.

Verified: every one of the 6 depth-check values (across 4 tests, two of which check both an intermediate and a final depth) independently re-derived from real arity and confirmed correct; every real word reference (`TOR`/`FROMR`/`RFETCH`/ `TWOTOR`/`TWOFROMR`/`TWORFETCH`) confirmed to resolve, with the apparent "missing" list from the check itself traced to false positives (register names, a mnemonic omitted from the exclusion list, and this batch's own local `FAIL`/`DONE` labels) rather than dismissed without checking; all 4 tests confirmed wired into `TSTRETSTACK`. Full scenario matrix re-run with `TSTSELECTOR`'s eighth value included (18 scenarios total) - all pass; explicitly confirmed `TSTSELECTOR=7` includes only this group's own bodies, with `TSTSTACK`'s and `TSTCTRLFLOW`'s bodies genuinely absent. Byte-exact split-file reassembly

```
confirmed, including the correct glossary-order placement (
confirmed via label position, not just presence).
**MAME-CONFIRMED**: the user reports all `TSTRETSTACK` tests
pass - including both round-trip intermediate checks and both
non-destructive-peek verifications - confirming every `>R`/
`2>R` in this group stayed correctly balanced against a
matching `R>`/`2R>` on real hardware, with no corruption of the
return path back out of this test group.
```

- ** `TSTSYSIO` added - System/Console I/O tests (glossary section 3.1, 12 words, 7 tests), wired into `TSTRUNNER` and inserted **first** in the source, ahead of `TSTSTACK`, per explicit request - matching this session's established practice of placing new groups in glossary order in the source even while `TSTSELECTOR`'s own numbering stays non-sequential for now (gated as its ninth value, `-8`, the next available number).

  ```
  **Six of the section's 12 words are deliberately not tested,
  confirmed necessary by reading their own implementations
  directly, not assumed from the glossary descriptions alone**:
  `KEY` genuinely spins forever without real input (`BEQ KEY`
  branches back to itself); `ACCEPT` calls `KEY` directly in its
  own main loop, and `EXPECT`/`QUERY` both call `ACCEPT` - all
  four inherit the same block, and none can complete during
  automated, headless boot-time testing where no real input will
  ever arrive. `ABORT` falls straight through into `QUIT`, which
  resets the return stack (`S`) to `RP0` - discarding this test
  framework's *entire* call chain and hijacking the boot sequence
  into the interpreter's own top-level loop; calling either
  directly would break the whole boot, not just fail one test.
  This is a materially different situation from every prior
  section's own deliberate coverage gaps (`DO`'s `limit=index`
  case, bare `CREATE`) - those were single edge cases within an
  otherwise-testable word; here, half the section's words are
  fundamentally incompatible with this framework's own execution
  model, not just one case of each.

  The other 6 (`KEY?`, `EMIT`, `TYPE`, `CR`, `SPACE`, `SPACES`,
  the last split into two tests for its `n<=0` no-op case) are
  confirmed safe - and already proven so, since this whole test
  framework has used `TYPE`/`CR` (via `TSTREPORT`) thousands of
  times already across every prior test group without incident.
  Each test here verifies the real output-queuing mechanism
  (`OUTHEAD`/`OUTBUF`) directly - reading the actual character(s)
  back out of the output ring buffer to confirm they were
  genuinely queued, not just that the call returned without
  crashing.
  ```

Two `FCB`-length/string mismatches caught by a programmatic check during writing, not left to be found later: `TSTSPACESZ` and `TSTSPACES`'s own name-length bytes were miscounted by hand (11 vs the true 10, 10 vs the true 9) - found by systematically re-checking every declared length against the actual string for every test in this batch, not just the ones that happened to look suspicious.

Label collisions checked programmatically before insertion - caught 3 real ones this way (`KQFALSE`, `SPDONE`, `TYDONE`, all internal labels already inside `KEY?`'s, `SPACES`'s, and `TYPE`'s own real implementations - the same class of issue as `MSTAR`/`LSHIFT`/`RSHIFT`/`HOLDS`/`DUMP` found in earlier groups), replaced with `KY`, `SC`, `TE` respectively, each re-verified against every label in the file.

The same missing-second-`ENDC` mistake as sections 3.3 and 3.8's own first attempts, and a separately forgotten `TSTRUNNER` wire-up - both caught and fixed before delivery: the `IFEQ`/`ENDC` balance check (run immediately after insertion, per this session's now-standard practice) caught the missing `ENDC` right away; a follow-on check specifically confirming `TSTSYSIO` appears in `TSTRUNNER`'s own call list (not just assumed done from memory) caught that it had been omitted entirely.

Verified: every one of the 7 depth-check values independently re-derived from real arity and confirmed correct against the final, fully-assembled file state (not just the pre-insertion draft); every real word reference (`KEYQ`/`EMIT`/`CRW`/ `SPACEW`/`SPACESW`/`TYPE`) confirmed to resolve, including `CRW` specifically, which an over-aggressive local-prefix filter in the verification script itself initially hid - confirmed directly via a literal `JSR CRW` count rather than trusting the flawed automated check; all 7 tests confirmed wired into `TSTSYSIO`, and `TSTSYSIO` itself confirmed wired into `TSTRUNNER`. Full scenario matrix re-run with `TSTSELECTOR`'s ninth value included (20 scenarios total) - all pass; explicitly confirmed `TSTSELECTOR=8` includes only this group's own bodies. Byte-exact split-file reassembly confirmed, including the correct glossary-order placement ahead of `TSTSTACK` (confirmed via label position).
MAME-CONFIRMED: the user reports all `TSTSYSIO` tests now pass under the real, active `SERIALPOLL=1` build - including the terminal output itself visibly showing each test's own emitted character(s) (`A` before `TSTEMIT`, a leading space

```
before `TSTSPACE`, three before `TSTSPACES`, `AB` before
`TSTTYPE`) intermixed with the pass/fail reporting exactly as
expected, confirming both the `EMITCH`-based and the
`SERIALPOLL`-conditional fixes hold correctly on real hardware,
not just in simulation.
```

- **RESOLVED - real bug found by the user's actual MAME run, not caught by this session's own verification: 5 of `TSTSYSIO` 's 7 tests failed, despite the characters genuinely appearing in the terminal output.** Root cause confirmed by directly re-checking `EMIT` 's own code, having *already found this exact pattern* once already for `KEY` earlier in the same investigation and failing to check whether it applied to `EMIT` too: `EMIT` has two entirely different implementations, gated by `SERIALPOLL` the same way `KEY` 's own two variants are - the interrupt-driven one (`SERIALPOLL=0`) queues into a ring buffer (`OUTHEAD` / `OUTBUF`), but the polling one (`SERIALPOLL=1` , confirmed the currently active build) writes directly to `ACIADR` and never touches `OUTHEAD` / `OUTBUF` at all. Every one of these 5 tests assumed the interrupt-driven mechanism unconditionally - so the checks always failed under the actual active build, despite `EMIT` genuinely working (the character really was transmitted, exactly as the user's own terminal output showed).

  ```
  Fixed with a mix of two approaches, chosen per test based on
  what's genuinely verifiable in each mode: `TSTEMIT`/`TSTSPACE`
  (single-`EMIT`-call tests) now check `EMITCH` universally, with
  no `SERIALPOLL` conditional needed at all - confirmed by reading
  both `EMIT` variants that `EMITCH` is set unconditionally, as
  the very first step, by both, before either mode-specific branch
  even begins. `TSTCR`/`TSTSPACES`/`TSTTYPE` (multiple-`EMIT`-call
  tests, where `EMITCH` only reflects the *last* call) now use an
  explicit `IFEQ SERIALPOLL`/`ELSE`/`ENDC` split: the full
  `OUTHEAD`/`OUTBUF` check for interrupt-driven mode, and an
  honest, narrower check (last character via `EMITCH`, plus the
  pre-existing stack-depth check) for polling mode - a genuine,
  accepted limitation of a direct hardware write with no
  buffering to inspect, not something worth contorting around.

  **A second, structural mistake caught and fixed during this same
  round, before delivery**: the first attempt put `IFEQ
  SERIALPOLL` directly on the same source line as `TSTCR:`/
  `TSTSPACES:`'s own label (`TSTCR:   IFEQ SERIALPOLL`) - a
  pattern that appears nowhere else in this entire file, where
  every other `IFEQ` sits on its own dedicated line. This confused
  this session's own `IFEQ`/`ENDC` balance-checking script (whose
  regex requires whitespace, not a label, before `IFEQ`), which is
  itself just a verification-script limitation - but rather than
  ```

```
trust that the real assembler would handle the untested pattern
correctly, restructured both to put the label and `IFEQ` on
separate lines, matching the proven convention used everywhere
else in this file. Re-verified via an explicit, full-file,
line-by-line nesting trace (not just a raw open/close count
match, which can't by itself distinguish correct nesting from a
coincidentally-equal but wrongly-ordered mismatch) that the
result is genuinely, correctly balanced end to end, with zero
negative-depth excursions anywhere in the file.

Verified: all 7 depth-check values re-confirmed unchanged by
these purely corrective edits, using a more robust per-test
isolation method after the original 200-character-window
heuristic in this session's own check turned out too narrow
once the new `SERIALPOLL` blocks pushed some tests' own depth
captures further from their label; explicitly confirmed via
simulation that `TSTCR`'s two branches genuinely diverge under
each `SERIALPOLL` setting (`OUTHEAD` check present only when
`SERIALPOLL=0`, `EMITCH` check present only when `SERIALPOLL=1`,
mutually exclusive as intended) rather than assumed correct from
the source alone. Full scenario matrix re-run crossing both real
`SERIALPOLL` settings with every `TSTSELECTOR` value (20
scenarios total) - all pass. Byte-exact split-file reassembly
confirmed. This specific fix awaits its own MAME confirmation -
the user's own report and register/memory-level detail is what
caught the root cause; this response addresses exactly what was
found.
```

- **RESOLVED - `TSTSELECTOR` 's comparison values renumbered sequentially starting at 0, matching glossary/source order, per explicit request.** The prior numbering had grown ad hoc across nine separate additions (each new group taking "next available" rather than matching its glossary-order position, per earlier explicit direction deferring exactly this renumbering to a later request). New mapping, all confirmed matching each group's own source position (which already matched glossary order): `TSTSYSIO` (3.1) `8` -> `0` , `TSTSTACK` (3.2) `0` -> `1` , `TSTRETSTACK` (3.3) `7` -> `2` , `TSTSARITH` (3.4) `1` -> `3` , `TSTDARITH` (3.5) `2` -> `4` , `TSTLOGIC` (3.6) `3` -> `5` , `TSTCOMPARE` (3.7) `4` -> `6` , `TSTCTRLFLOW` (3.8) `5` -> `7` , `TSTDEFWORDS` (3.9) `6` -> `8` .

  ```
  Applied via a position-bounded replacement, not a blanket find-
  replace across the whole file - each group's own two `IFEQ
  TSTSELECTOR-N` occurrences were located and replaced only within
  that specific group's own source region (bounded by its own
  label through the next group's), avoiding any risk of a
  sequential renumber colliding mid-process (e.g. an already-
  renumbered `-1` accidentally being renumbered again when a
  ```

```
different group's original `-1` gets processed later). All 18
replacements (2 per group x 9 groups) confirmed exact.

Checked the rest of the file for stale comment references to
specific `TSTSELECTOR` values before considering this complete -
found none needing correction: the one literal `TSTSELECTOR=2`
appearing in the source is a generic syntax example in the unit-
test framework's own header comment, not tied to a specific
group's identity, and left as-is. The checklist's own historical
entries documenting each group's original, non-sequential value
at the time it was added are left untouched as well, being a
chronological record of what was actually done, not a live,
always-current description of the file's present state.

Verified: `IFEQ`/`ENDC` balance unaffected (31/31, unchanged
from before - this edit only changed values within existing
pairs, not the pairs themselves) and correct nesting re-
confirmed via a full-file line-by-line trace; full scenario
matrix re-run crossing both real `SERIALPOLL` settings with
`TSTSELECTOR` 0 through 8 (20 scenarios total) - all pass;
explicitly confirmed via simulation, not just inferred from the
source text, that every `TSTSELECTOR` value 0-8 now compiles
exactly the test group matching its own glossary-section
ordinal (a representative test spot-checked per group). Byte-
exact split-file reassembly confirmed. Not yet confirmed via
MAME, though given this touched only which literal number
selects which already-working group - not any group's own logic
- the risk profile is low relative to this session's typical
finds.
```

- ** `TSTCOMPWORDS` added - compiling-words tests (glossary section 3.10, 13 words, 17 tests since several get separate cases: `'` found/not-found, `[']` compiling/interpreting state, `POSTPONE` normal/immediate word, `SLITERAL` compiling/ interpreting state, `ABORT"` false/true flag), wired into `TSTRUNNER` and inserted last in the source (after `TSTDEFWORDS`), matching glossary order - the first new group whose `TSTSELECTOR` value (`-9`) matches its glossary-order position from the moment it was added, since the prior renumbering (above) already made every earlier group's value sequential.

```
**`POSTPONE` confirmed this section's hardest case, as
anticipated**: a genuine two-level compile-time mechanism.
Traced its own code first rather than assumed from the
glossary's one-line description - for a normal (non-immediate)
word, it compiles `[LIT xt][JSR COMPILE,]` into the current
definition, not a direct call - standard ANS semantics
(`POSTPONE` appends a word's own *compilation* semantics to the
```

current definition; for an ordinary word that semantics IS "compile a call to it", so appending it only makes sense if the current definition, when it later runs, itself performs that compiling action against whatever is being compiled at that later point - not immediately). For an immediate word, by contrast, it compiles a direct call to the immediate word's own xt - simpler, confirmed by reading `FIND`'s own code that it pushes `1` for immediate words, `-1` otherwise, and `POSTPONE` branches on exactly that. Given the genuine complexity of fully executing the normal-word case's second level (which would need a further redirected `CODEHERE` and a further round of execute-and-verify), both tests instead verify the compiled byte sequence directly - still a meaningful, unambiguous check of the actual mechanism, not a workaround.

`ABORT"` confirmed safely testable for both flag cases, not just assumed from its relationship to `ABORT`: traced `DOABORTQUOTE`'s own runtime code directly and confirmed the true-flag path uses `THROW -2` internally, not the raw, never-returns `ABORT` mechanism plain `ABORT` itself uses (still deliberately untested, per section 3.1's own reasoning - `ABORT` resets the return stack and would hijack this whole test framework's boot sequence). Since `ABORT"`'s own exception is genuinely catchable, both its false-flag (resumes normally, confirmed by checking execution lands exactly past the inline message) and true-flag (`CATCH`-wrapped, confirms `-2`) cases are tested directly.

`LITERAL` and `>BODY` had already been used extensively as internal helpers in sections 3.8/3.9's own tests, but got dedicated, direct tests here too, per this section's own coverage - each confirms genuine end-to-end behavior (executing a compiled snippet, or computing a real offset), not just reconfirming what their existing indirect use already implied.

Three `FCB`-length/string-length mismatches caught by the now-standard programmatic check during writing, not left to be found later: `TSTCOMPCOMMA` (declared 13, actually 12), `TSTPOSTPONE1` (declared 13, actually 12), `TSTBRACKTICK1`/ `TSTBRACKTICK2` (each declared 14, actually 13), `TSTSLITERAL1`/ `TSTSLITERAL2` (each declared 13, actually 12) - checked every single test's own name length individually and immediately after writing it, rather than batching the check to the end, given how frequently this specific mistake had already recurred across earlier sections.

Label collisions checked programmatically before insertion -

```
caught 2 real ones this way (`LTFAIL`/`LTDONE`, colliding with
section 3.7's own `TSTLT`; `EXFAIL`/`EXDONE`, colliding with
section 3.8's own `TSTEXIT`), replaced with `LI` and `XQ`
respectively, each re-verified against every label in the file.
Checked the `IFEQ`/`ENDC` balance immediately after insertion,
per this session's now-standard practice following the repeated
missing-second-`ENDC` mistake in earlier sections - balanced
(33/33) and correctly nested end to end on the first attempt
this time.

Verified: every one of the 17 depth-check values independently
re-derived from real arity and confirmed correct against the
final, fully-assembled file state; every real word reference
confirmed to resolve, with the apparent "missing" list (a
handful of character-literal tokens, a mnemonic omitted from
the exclusion list, and this batch's own local labels) traced to
its real, harmless cause rather than dismissed without checking;
all 17 tests confirmed wired into `TSTCOMPWORDS`, and
`TSTCOMPWORDS` itself confirmed wired into `TSTRUNNER`. Full
scenario matrix re-run with `TSTSELECTOR`'s tenth value included
(22 scenarios total) - all pass; explicitly confirmed
`TSTSELECTOR=9` includes only this group's own bodies, with
`TSTDEFWORDS`'s and `TSTSYSIO`'s bodies genuinely absent. Byte-
exact split-file reassembly confirmed. **MAME-CONFIRMED**: the
user reports all `TSTCOMPWORDS` tests now pass, including
`TSTTOBODY` after its own fix (logged separately above) - the
`POSTPONE` normal/immediate-word cases, both `ABORT"` flag
cases, and every other test in this section all confirmed
correct on real hardware, not just in simulation.
```

- **RESOLVED - real bug found by the user's actual MAME run, pinpointed precisely from a register/memory dump: `TSTTOBODY` failed because the test's own assumption about `TOBODY` 's return value was wrong - `TOBODY` itself is correct.** Root cause confirmed by re-reading `TOBODY` 's own code directly rather than trusting the earlier trace: it computes `xt+5` , then *dereferences* it (`LDD ,X`), returning the value stored there - not `xt+5` itself. This test's own comment had described it as "a pure, fixed-offset computation… not dependent on what's actually at that offset" - simply wrong, and the test never initialized `TSTCBUF+5` to anything, so it compared the real returned value against whatever garbage happened to already be sitting there ($BDEB in the user's own register dump, exactly matching the two bytes shown at that address in the accompanying memory dump - not a coincidence, the precise mechanism the trace predicted).

```
Reconciled this directly against why section 3.9's `DEFER@`/
```

`DEFER!`/`TO`/`IS` tests, which also rely on this exact same `TOBODY` and already passed on real hardware, never surfaced this: those tests set up real trampolines (via `VALUEW`/`DEFERW`), whose own compile-time code stores a genuinely meaningful value at `xt+5` (for `DEFER`, the current target xt; for `VALUE`, the stored value itself) - for this system's own trampoline design, "the body" *is* the value stored at `xt+5`, not `xt+5`'s own address. `TOBODY` was never broken; this one test's own understanding of what it does was.

Fixed by setting a known value at `TSTCBUF+5` before calling `TOBODY`, and verifying it returns exactly that value - not the address. Also corrected the test's own header comment, which had baked in the same wrong assumption.

Verified: `IFEQ`/`ENDC` balance unaffected (33/33); this test's own depth-check value unchanged (`0`, since the fix only adds a memory write before the stack-tracking baseline, touching nothing on `U`); full scenario matrix re-run crossing both real `SERIALPOLL` settings with every `TSTSELECTOR` value 0-9 (22 scenarios total) - all pass. Byte-exact split-file reassembly confirmed. This specific fix awaits its own MAME confirmation - the user's own register/memory-dump-level detail is what let this be pinpointed precisely rather than guessed at.

- ** `TSTMEMORY` added - memory tests (glossary section 3.11, 22 words, 16 tests since several are combined into round-trip tests that can only be meaningfully verified together - `@` / `!` , `C@` / `C!` , `2@` / `2!` , and the `CODEHERE` -region words (, / `C`, / `ALLOT` / `HERE`) and `VARHERE` -region words (`V`, / `VC`, / `VALLOT` / `VHERE`) each combined into one sequential walk-through per region), wired into `TSTRUNNER` and inserted last in the source (after `TSTCOMPWORDS`), matching glossary order, gated as `TSTSELECTOR` 's eleventh value (`-10`) - continuing the now- sequential numbering established by the earlier renumbering.

  Mostly simpler, direct memory-access words than the last few sections (no compile-time behavior, no name-parsing), but `,`/`C,`/`ALLOT`/`HERE`/`PAD` touch `CODEHERE` and `V,`/`VC,`/`VALLOT`/`VHERE` touch `VARHERE`, so those still needed the same redirect-based safety established for sections 3.8/3.9; `TSTCBUF`/`TSTVBUF` reused as general scratch for the simpler fetch/store tests too, safe since each `TSTSELECTOR` group runs exclusively of every other.

  `MOVE` confirmed via its own code to genuinely choose its copy direction by comparing addresses, delegating to `CMOVE`

(low-to-high) when `dst <= src` and `CMOVE>` (high-to-low) otherwise, specifically to stay overlap-safe. Rather than test only a trivial non-overlapping case, wrote two dedicated overlapping-region tests (`TSTMOVE2`/`TSTMOVE3`, one for each direction) - each expected result hand-derived byte by byte and then independently re-derived via a small Python simulation before writing the actual test, not guessed at or assumed correct from the reasoning alone. `CMOVE`/`CMOVE>` themselves are tested with non-overlapping regions only, given they are explicitly documented as not overlap-safe in one direction each - `MOVE`'s own existence is precisely the mechanism for handling the overlapping case correctly, not something to duplicate in `CMOVE`'s own tests.

`TSTCFETCHSTORE` (`C@`/`C!`) specifically pre-fills the cell with a distinctive nonzero high byte before testing, to verify both that `C!` genuinely only touches the low byte (checked via a direct memory read, not just inferred from the later `C@`) and that `C@` genuinely zero-extends (not just returns whatever was already there) - a plain round trip alone could not have distinguished either from a byte-store/byte-fetch pair that happened to leave the high byte untouched by coincidence.

Zero `FCB`-length/string-length mismatches survived to the collision-check stage this time - all nine caught (`TSTFETCHSTORE`, `TSTPLUSSTORE`, `TSTPAD`, `TSTUNUSED`, `TSTVUNUSED`, `TSTCMOVE`, `TSTCMOVEGT`, and two others) individually and immediately after writing each test, per this session's now-standard practice. Label collisions checked programmatically before insertion - caught 4 real ones this way (`PDFAIL`/`PDDONE`, colliding with section 3.5's own `TSTDPLUS`; `CMDONE`/`CGDONE`, colliding with `CMOVEW`'s/`CMOVEGT`'s own internal completion labels - the same class of issue as `MSTAR`/`LSHIFT`/`RSHIFT`/`HOLDS`/`DUMP`/`TYPE` found in earlier sections), replaced with `PA`/`CV`/`CX` respectively, each re-verified against every label in the file, including this batch's own internal loop labels.

Checked the `IFEQ`/`ENDC` balance immediately after insertion, per this session's now-standard practice - balanced (35/35) and correctly nested end to end on the first attempt.

Verified: all 16 depth-check values independently re-derived from real arity and confirmed correct against the final, fully-assembled file state; every real word reference confirmed to resolve, with the apparent "missing" list (character literals, hex literals, a mnemonic, and this batch's own local loop labels) traced to its real, harmless cause rather than dismissed

```
without checking; all 16 tests confirmed wired into `TSTMEMORY`,
and `TSTMEMORY` itself confirmed wired into `TSTRUNNER`. Full
scenario matrix re-run with `TSTSELECTOR`'s eleventh value
included (24 scenarios total) - all pass; explicitly confirmed
`TSTSELECTOR=10` includes only this group's own bodies. Byte-
exact split-file reassembly confirmed. **MAME-CONFIRMED**: the
user reports all `TSTMEMORY` tests now pass, including
`TSTCFETCHSTORE` after its own fix (logged separately above) -
both overlapping-direction `MOVE` cases, the combined `CODEHERE`/
`VARHERE`-region walk-throughs, and every other test in this
section all confirmed correct on real hardware.
```

- **RESOLVED - real bug found by the user's actual MAME run: `TSTCFETCHSTORE` failed because the test's own expected value after `C!` was backwards - `CSTOREW` itself is correct.** Root cause confirmed directly against the user's own register and memory dump: this test expected `$FF34` after storing `$34` into a cell pre-filled with `$FFFF`, assuming the untouched byte would end up as the word's high byte - but `CSTOREW` writes directly at the exact address given (`TSTCBUF` itself, the *lower* address), leaving `TSTCBUF+1` (the higher address) untouched at `$FF`. Since `LDD` on this big-endian 6809 reads the word's high byte from the lower address first, the correct result is `$34FF`, not `$FF34` - exactly what the user's own dump showed (`D=$34FF` at the failing `CMPD`, with the memory dump confirming `34 FF` sitting at `TSTCBUF` / `TSTCBUF+1`). The single-byte write and the zero-extending fetch this test also checks were both already correct; only this one comparison's own arithmetic about which byte ends up where was wrong.

  ```
  Fixed by correcting the expected value to `$34FF` and rewriting
  the test's own header comment, which had described the pre-fill
  byte's position ambiguously enough to have contributed to the
  original mistake - now states precisely that `C!` touches the
  single byte at the exact address given, not "the other half" of
  a 2-byte cell in some assumed sense.

  Verified: `IFEQ`/`ENDC` balance unaffected (35/35); this test's
  own depth-check value unchanged (`-2`, since the fix only
  changes a comparison constant); full scenario matrix re-run
  crossing both real `SERIALPOLL` settings with every
  `TSTSELECTOR` value 0-10 (24 scenarios total) - all pass. Byte-
  exact split-file reassembly confirmed. This specific fix awaits
  its own MAME confirmation - the user's own register/memory-dump-
  level detail is what let this be pinpointed precisely rather
  than guessed at.
  ```

- ** `TSTSTRPARSE` added - strings & parsing tests (glossary section 3.12, 16 words, 19

tests since several get separate cases: `[CHAR]` compiling/interpreting state, `S"` compiling/ interpreting state (this word genuinely has both), `SEARCH` found/not-found, `REPLACES` / `SUBSTITUTE` combined across two tests, `SNAME` found/not-found), wired into `TSTRUNNER` and inserted last in the source (after `TSTMEMORY`), matching glossary order, gated as `TSTSELECTOR` 's twelfth value (`-11`).

```
**A real, significant naming collision caught by the standard
collision check, not just an internal-label clash like earlier
sections' finds**: this section's own `COMPARE` (string
comparison) word's test was originally named `TSTCOMPARE` -
which turned out to already be the group-level wrapper name for
section 3.7's entire comparison-operators test group (`=`, `<`,
`>`, etc. - a completely different, pre-existing thing, not an
internal label inside some word's own implementation). Renamed
to `TSTSCOMPARE` throughout, including its own display name (the
string `TSTREPORT` prints), confirmed via simulation that the
real section 3.7 `TSTCOMPARE` wrapper is completely unaffected
and still compiles correctly, and confirmed `TSTSCOMPARE` itself
doesn't collide with anything either. Three more internal-label
collisions caught the same way as earlier sections (`PNDONE`
inside `PARSENAME`'s own real implementation, `UEDONE` inside
`UNESCAPEW`'s own real implementation, `DQFAIL`/`DQDONE`
colliding with an existing section 3.7 test's own labels),
replaced with `PZ`/`UX`/`DX` respectively.

**`.` confirmed to have no `STATE` check at all in its own
code** - matches "no interpretation semantics, per ANS" without
a specific `-14` throw (unlike `[CHAR]`/`S"`, which do throw),
so only its compiling case is tested, matching how it's
actually meant to be used. Its own runtime (`DOTSTR`) calls
`TYPE` internally, which - per section 3.1's own already-
debugged finding - has two entirely different implementations
gated by `SERIALPOLL`; this test applies that same lesson from
the start rather than re-discovering it via a failed MAME run,
using the same split established for `TSTCR`: full `OUTHEAD`/
`OUTBUF` verification under `SERIALPOLL=0`, a narrower `EMITCH`-
based check under `SERIALPOLL=1`.

**`PARSE` and `PARSE-NAME` each get a test specifically designed
to isolate their one documented behavioral difference** - a fake
source starting with the delimiter itself, confirming `PARSE`
genuinely does not skip it (returns a zero-length token
immediately) while `PARSE-NAME` genuinely does skip leading
spaces - rather than two generic parse tests that could pass
even if the two words were accidentally sharing one code path.
```

`SUBSTITUTE`'s own expected outputs were independently simulated in Python against the algorithm as traced from the real source, not hand-derived and assumed correct - covering all 4 of its own documented `%`-delimiter cases across two tests: `TSTREPLSUBS1` (the `%%`-collapses-to-`%` case, a registered-name substitution, and a non-registered-name pass-through, all in one template) and `TSTREPLSUBS2` (an unpaired trailing `%` with no closing delimiter, tested in isolation for clearer failure diagnosis). `UNESCAPE`'s and `REPLACES`'s own extensive prior-session bug-fix comments (doubling `%`, not backslash-escaping; single-slot registration) were confirmed fresh against the actual current code before designing tests around them, not trusted blindly.

Nine `FCB`-length/string-length mismatches caught individually and immediately after writing each test, per this session's now-standard practice (`TSTCHARW`, `TSTPARSE`, `TSTPARSENAME`, `TSTBRACKCHAR1`/`TSTBRACKCHAR2`, `TSTDASHTRAILING`, `TSTSLASHSTRING`, `TSTREPLSUBS1`, `TSTSCOMPARE` after its own rename left a stale length byte from the old, shorter name).

Checked the `IFEQ`/`ENDC` balance immediately after insertion, per this session's now-standard practice - balanced (39/39) and correctly nested end to end on the first attempt.

Verified: all 19 depth-check values independently re-derived from real arity and confirmed correct against the final, fully-assembled file state; every real word reference confirmed to resolve, with the apparent "missing" list (character literals, mnemonics, directives, and this batch's own local labels) traced to its real, harmless cause rather than dismissed without checking; all 19 tests confirmed wired into `TSTSTRPARSE`, and `TSTSTRPARSE` itself confirmed wired into `TSTRUNNER`. Full scenario matrix re-run with `TSTSELECTOR`'s twelfth value included (26 scenarios total) - all pass; explicitly confirmed `TSTSELECTOR=11` includes only this group's own bodies, and separately confirmed via simulation that the real section 3.7 `TSTCOMPARE` group's own body is still present and unaffected by this section's rename. Byte-exact split-file reassembly confirmed. **MAME-CONFIRMED**: the user reports all `TSTSTRPARSE` tests pass, including `TSTDOTQUOTE` - the terminal output itself visibly showing "HI" (the test's own emitted string) ahead of "TSTDOTQUOTE OK", confirming the `SERIALPOLL`-conditional handling applied from the start (per section 3.1's own lesson) held correctly on real hardware on the first attempt, with no repeat of that earlier bug. Both `S"` state cases, both `[CHAR]` state cases, `SUBSTITUTE`'s

```
independently-simulated expected outputs, and the renamed
`TSTSCOMPARE` (confirmed not colliding with the real section 3.7
group) all confirmed correct as well.
```

- ** `TSTNUMOUT` added - numeric output tests (glossary section 3.13, 14 words, 13 tests since `#S` and `#>` are combined into one test, matching how they're naturally used together as the tail of the standard pictured-output idiom `<# ... #S #>`), wired into `TSTRUNNER` and inserted last in the source (after `TSTSTRPARSE`), matching glossary order, gated as `TSTSELECTOR`'s thirteenth value (`-12`).

```
**`SIGN` confirmed to carry real bug-fix history worth designing
the test around, not just noting**: a prior version used a bare
`BPL` immediately after `PULU`, which doesn't affect condition
codes on real 6809 hardware - the bug was masked for every
internal caller (DOT/DOTR/DDOT/DDOTR) by "caller-side luck" (each
happens to run a flag-setting `LDD` of the true value right
before calling `SIGN`, and `PSHU` doesn't disturb flags). The
current code is fixed (explicit `TSTA` immediately before the
branch), but a naive test calling `SIGN` the same way those
internal callers do would repeat the identical caller-side-luck
pattern and fail to catch a future regression if `TSTA` were
ever removed again. This section's own `TSTSIGN` instead
deliberately poisons the CPU flags with an unrelated `CMPX`
(setting them to reflect an unrelated comparison result, without
touching the value about to be pushed) immediately before each
direct `PSHU`/`JSR SIGN` call, for both a negative and a positive
test value - genuinely exercising the "direct, standalone call"
scenario the bug-fix comment says was previously broken, not a
scenario likely to pass by the same kind of luck that hid the
original bug.

**Every one of the 7 high-level printing words (`.`/`U.`/`.R`/
`U.R`/`?`/`D.`/`D.R`) applies the `SERIALPOLL`-conditional split
established in section 3.1 from the start** - full `OUTHEAD`/
`OUTBUF` verification under `SERIALPOLL=0`, an `EMITCH`-based
last-character check under `SERIALPOLL=1` - rather than
discovering the need for it via a failed MAME run, as originally
happened in section 3.1 itself. Explicitly confirmed via
simulation (not just assumed from the source) that both branches
genuinely compile and diverge as intended for a representative
test. Accepted, documented limitation carried over honestly: for
the three words with a fixed trailing space (`.`/`U.`/`D.`), the
polling-mode last-character check only confirms a space was
emitted, not the actual printed digits - the same trade-off
already established for `TSTCR`/`TSTSPACES`/`TSTTYPE`/
```

`TSTDOTQUOTE` in earlier sections, not a new one invented here. `.R`/`U.R`/`D.R` specifically test the padding path (a width wider than the actual digit count - "42" with width 5 producing 3 leading spaces), not just a no-padding sanity case. `D.`/`D.R` use a genuine double-cell value (-100000: high cell `$FFFE`, low cell `$7960`) computed and independently verified in Python before writing the tests, specifically to exercise the 32-bit negation path (`MNEG32`), not a value that happens to fit in a single cell.

Caught and fixed a real mistake in this section's own first `TSTSIGN` draft before it was ever inserted: an intermediate check compared `HLD` against `CODEHERE`'s raw redirected value instead of the actual computed `PAD` address (`CODEHERE` plus `PADOFFSET`) - would have been silently wrong every time, not caught by any later verification step since it was fixed during drafting, before the collision/depth/reference checks ran.

A significant naming-collision risk from the prior section (`TSTCOMPARE` accidentally colliding with an existing group wrapper) prompted extra care here: this group's own wrapper name (`TSTNUMOUT`) was explicitly checked against the whole file before use, not assumed safe by pattern alone. Five internal-label collisions still turned up via the standard check and were fixed the same way as every prior section's finds: `HSFAIL`/`HSDONE` (the latter colliding with `HOLDS`' own real internal completion label), `NSFAIL`/`NSDONE`, `NGFAIL`/`NGDONE`, `UDFAIL`/`UDDONE` (three colliding with existing, unrelated tests' own labels from earlier sections), replaced with `HO`, `NZ`, `NX`, `UF` respectively.

Seven `FCB`-length/string-length mismatches caught individually and immediately after writing each test, per this session's now-standard practice.

Checked the `IFEQ`/`ENDC` balance immediately after insertion - balanced (55/55, an increase of exactly 16 pairs over the prior section's 39, matching the expected count precisely: 7 I/O tests x 2 `SERIALPOLL` blocks each, plus 2 group-level pairs) and correctly nested end to end on the first attempt.

Verified: all 13 depth-check values independently re-derived from real arity and confirmed correct against the final, fully-assembled file state; every real word reference confirmed to resolve, including several (`QMARK`, `DDOT`, `DDOTR`, `HOLD`, `HOLDS`) initially hidden from the automated check by this session's own recurring local-prefix-exclusion artifact, each

```
confirmed directly via a literal call-site count rather than
trusted blindly; all 13 tests confirmed wired into `TSTNUMOUT`,
and `TSTNUMOUT` itself confirmed wired into `TSTRUNNER`. Full
scenario matrix re-run with `TSTSELECTOR`'s thirteenth value
included (28 scenarios total) - all pass; explicitly confirmed
`TSTSELECTOR=12` includes only this group's own bodies. Byte-
exact split-file reassembly confirmed. **MAME-CONFIRMED**: the
user reports all `TSTNUMOUT` tests now pass, including
`TSTNUMSIGN`/`TSTNUMSIGNSGT` after their own two fixes (the
pre-`COLD` `BASE=0` infinite loop and the depth-check mistake,
both logged separately above) - the deliberately-poisoned-flags
`TSTSIGN` test, both padding-path tests (`.R`/`U.R`), and the
genuine double-cell negation in `D.`/`D.R` all confirmed correct
on real hardware, with the terminal output itself visibly
showing each printed value ("-42", "42", "   42", "-7",
"-100000", "   -100000") exactly as designed.
```

- **RESOLVED - real bug found by the user's actual MAME run: an infinite loop in `TSTNUMSIGNSGT`, which turned out to affect every test in this section that does digit conversion, not just the one that happened to surface it.** Root cause confirmed by tracing the real boot sequence directly, not assumed: this entire test framework runs from `COLDSTRT`, which clears the whole `GLOBALS` region (including `BASE`, at offset `$02`) to zero and *then* calls `TSTRUNNER` - with `COLD`'s own `BASE=10` initialization not running until later, after the test framework has already finished. So `BASE` reads as genuine zero at test time, not 10. With `BASE=0`, `UDDIGIT`'s own restoring- division algorithm (traced precisely against the user's own register/memory dump, which showed `REM`/`UDHI`/`UDLO` all stuck at `$FFFF`/`$FF` after `DCNT` had reached zero) degrades into an unconditional shift - the "compare against `BASE`, then subtract" step can never actually subtract, since nothing can compare below zero - so the value being converted never genuinely decreases. `NUMSIGNS`' own loop-until-zero condition then never becomes true, and `HOLD` keeps decrementing without bound, exactly matching the user's own observation of memory filling with repeated `'6'` characters far beyond the pictured- output region's own bounds.

```
This was never a bug in `UDDIGIT`/`NUMSIGN`/`NUMSIGNS`
themselves, which reasonably assume `BASE` already holds a
valid, nonzero value by the time they're used - true for every
normal, post-`COLD` use of these words, just not true during
this framework's own pre-`COLD` execution window. Structurally
the same class of gap as every other system-variable dependency
already handled via save/redirect/restore throughout this whole
session (`CODEHERE`, `STATE`, `SRCADDR` etc.) - `BASE` was
simply the one variable in this dependency chain not yet
```

identified as needing the same treatment, since the words in this specific section were the first to genuinely depend on its value rather than just its address or general presence.

Fixed by adding an explicit save/set/restore of `BASE` (save the real value, set it to 10, restore afterward) to every one of the 9 affected tests: `TSTNUMSIGN`, `TSTNUMSIGNSGT`, `TSTDOT`, `TSTUDOT`, `TSTDOTR`, `TSTUDOTR`, `TSTQMARK`, `TSTDDOT`, `TSTDDOTR` - the same pattern already established for every other redirected system variable in this file, not a new mechanism invented for this fix. Added a new, dedicated scratch constant (`TSTBASAV`) for this, with a detailed comment explaining the root cause directly at its declaration, matching this session's established practice of documenting bug-fix history at the point of the fix, not just in this checklist. Applied the identical fix systematically across the 6 structurally-identical `SERIALPOLL`-conditional tests via a single Python transformation rather than six separate manual edits, then verified each one individually afterward (both the save/set point and the restore point) rather than trusting the bulk transformation blindly. Also cleaned up a cosmetic indentation inconsistency the bulk transformation left behind (a fixed 12-space indent on the inserted lines, not matching the surrounding code's own per-label indent width) before delivery, re-verifying balance and every depth check afterward to confirm the cleanup itself introduced no regressions.

Verified: `IFEQ`/`ENDC` balance unaffected (55/55, unchanged - this fix only added `LDD`/`STD` memory operations, no new conditional blocks); every one of the 13 depth-check values confirmed unchanged (the fix touches only `BASE`, never `U`); full scenario matrix re-run crossing both real `SERIALPOLL` settings with every `TSTSELECTOR` value 0-12 (28 scenarios total) - all pass, both before and after the indentation cleanup. Byte-exact split-file reassembly confirmed. This specific fix awaits its own MAME confirmation - the user's own register/memory-dump-level detail (specifically the `DCNT=0` reading, confirming the division's own internal 32-iteration loop had genuinely completed, combined with `REM`/`UDHI`/`UDLO` all reading `$FFFF`/`$FF`) is what allowed the root cause to be pinpointed to `BASE`'s own value precisely, rather than guessed at via the division algorithm's structure alone.

- **RESOLVED - real assembly error found by the user's actual lwasm run: the previous `BASE` fix's own cosmetic cleanup step introduced four column-1 mnemonics, each parsed by lwasm as a label rather than an instruction (producing**

both “Bad opcode” and, for the second occurrence at each pair, “Multiply defined symbol” - since `LDD` / `STD` as label names then collided across the two structurally-identical, previously-fixed tests). Root cause: the Python script used to normalize the earlier bulk fix’s own inconsistent indentation computed each inserted line’s indent from the immediately preceding line in the output - but at both of the user’s own reported locations (`TSTNUMSIGN` ’s and `TSTNUMSIGNSGT` ’s own `BASE` restore points), that preceding line was blank, so the computed indent came out as zero instead of matching the surrounding code.

```
Given the risk that this indentation-computation bug could have
produced other, unreported column-1 lines elsewhere in the file
(not just the two the user's own lwasm run happened to reach
before stopping), searched the entire file for every mnemonic
appearing at column 1, not just the two locations named in the
error report. Found two more, unrelated to this specific fix
and pre-dating this session's own `BASE` work entirely (`LDD
#10`/`PSHU D` pairs within `TSTCODEHERE`'s and `TSTVARHERE`'s
own `ALLOT`/`VALLOT` steps, both in section 3.11) - fixed all
four rather than only the two reported, then re-scanned the
entire file programmatically (checking every real 6809 mnemonic
against every line's own leading whitespace) to confirm zero
remaining column-1 occurrences anywhere, not just in the
sections touched by this specific round of fixes.

Verified: `IFEQ`/`ENDC` balance unaffected (55/55); every one
of the 13 depth-check values in this section confirmed
unchanged (these were purely whitespace fixes, touching no
logic); full scenario matrix re-run crossing both real
`SERIALPOLL` settings with every `TSTSELECTOR` value 0-12 (28
scenarios total) - all pass. Byte-exact split-file reassembly
confirmed. This specific fix awaits its own successful lwasm
assembly run to confirm - the user's own assembler error output
is what caught this, not a runtime/MAME-level issue.
```

- **RESOLVED - real bug found by the user’s actual MAME run: `TSTNUMSIGN` failed on its own final depth check - `NUMSIGN` itself is correct, the test’s own expected value was wrong.** Root cause confirmed directly against the user’s own register dump (`D=$0004` exactly at the failing `CMPD`): this test captures `TSTUAF` immediately after `NUMSIGN` returns, with `ud2` (the computed quotient) still sitting on the stack - 2 more cells than `TSTUB4` , which was captured before `NUMSIGN` even ran. The expected depth had been written as `0` , apparently by mistakenly copying the reasoning from the neighboring `TSTNUMSIGNSGT` (which calls `#>` to consume the accumulated result *before* capturing `TSTUAF` , genuinely landing back at net zero) without re-deriving it for this

test's own, structurally different capture point. Fixed to the correct value (`4`), with a comment at the fix explaining the mistake directly, matching this session's established practice.

```
Verified: `IFEQ`/`ENDC` balance unaffected (55/55, unchanged -
this was purely a comparison-constant fix plus an explanatory
comment); this test's own depth-check sequence re-confirmed
correct (`0`/`2`/`4`) using a more careful check that strips
comments first, after an initial naive regex check was fooled
by the new multi-line comment sitting between the fixed `CMPD`
and its own `BNE` (a verification-script limitation only - the
comment itself is standard, valid assembly syntax with no
bearing on real assembly or execution); confirmed zero column-1
mnemonics anywhere in the file, re-checked after this edit
specifically given the immediately preceding round of fixes in
this same file; full scenario matrix re-run crossing both real
`SERIALPOLL` settings with every `TSTSELECTOR` value 0-12 (28
scenarios total) - all pass. Byte-exact split-file reassembly
confirmed. This specific fix awaits its own MAME confirmation -
the user's own register-dump-level detail is what let this be
pinpointed to the exact, precise expected value rather than
re-derived from scratch or guessed at.
```

- ** `TSTBASERADIX` added - base/radix control tests (glossary section 3.14, 4 words: `BASE` / `DECIMAL` / `HEX` / `BINARY`), wired into `TSTRUNNER` and inserted last in the source (after `TSTNUMOUT`), matching glossary order, gated as `TSTSELECTOR` 's fourteenth value (`-13`). A small, simple section - all four words are pure `BASE` state-setting/reading with no I/O involved, so none of this section's own testing needed the `SERIALPOLL` -conditional complexity every printing-heavy section since 3.1 has required.

```
Combined into a single test (`TSTBASE`) rather than four
separate ones - each word's own effect is a single, trivially-
verified `BASE` read or write, so four separate tests would
just repeat the identical save/set/verify/restore shape four
times over with nothing meaningfully different to isolate.
Sets `BASE` via `HEX`/`BINARY`/`DECIMAL` in turn (verifying the
correct value lands each time), then calls `BASE` itself and
verifies both that it returns the variable's own address (not
its value, per its own documented `( -- addr )`) and that
reading through that returned address reflects `DECIMAL`'s own,
most recent setting - confirming the address is genuinely live,
not just structurally correct.

Reused `TSTBASAV` (the scratch constant added in section 3.13,
```

```
when the real, pre-`COLD` `BASE=0` bug was found and fixed) to
save and restore the real `BASE` across this test too, per the
same established reasoning - this section's own tests
deliberately set `BASE` to specific values as their whole point,
so leaving the real value disturbed afterward would be a
regression of exactly the kind that bug-fix was meant to
prevent going forward.

Checked the group wrapper name (`TSTBASERADIX`) against the
whole file before use, per the practice established after the
`TSTCOMPARE` collision in an earlier section - safe, no
collision. Checked every column-1 mnemonic occurrence
immediately after insertion too, per the practice established
after the indentation mistake found via a failed lwasm run in
section 3.13 - zero found.

Verified: `IFEQ`/`ENDC` balance immediately after insertion -
balanced (57/57) and correctly nested end to end on the first
attempt; the test's own depth-check value and all four
intermediate value checks confirmed correct; real word
reference confirmed to resolve for all four words; the test
confirmed wired into `TSTBASERADIX`, and `TSTBASERADIX` itself
confirmed wired into `TSTRUNNER`. Full scenario matrix re-run
with `TSTSELECTOR`'s fourteenth value included (30 scenarios
total) - all pass; explicitly confirmed `TSTSELECTOR=13`
includes only this group's own body. Byte-exact split-file
reassembly confirmed. **MAME-CONFIRMED**: the user reports
`TSTBASE` passes - `HEX`/`BINARY`/`DECIMAL` all confirmed
setting the real `BASE` correctly in sequence, and `BASE`
itself confirmed returning a genuinely live address, on real
hardware.
```

- ** `TSTEXCEPTION` added - exception handling tests (glossary section 3.15, 2 words: `CATCH` / `THROW`, 4 tests since the two can only be meaningfully tested together - `THROW`'s own effect is only observable through a `CATCH` that traps it), wired into `TSTRUNNER` and inserted last in the source (after `TSTBASERADIX`), matching glossary order, gated as `TSTSELECTOR`'s fifteenth value (`-14`).

```
**`CATCH`/`THROW` had already been used extensively as this
whole session's own error-testing mechanism throughout sections
3.8-3.14** - dozens of prior `-13`/`-14`/etc. throw-code checks
across earlier sections already give substantial indirect
evidence they work correctly - but this section still got its
own, direct, dedicated tests, matching the established practice
of every glossary section getting real coverage rather than
```

resting on incidental use elsewhere.

Critical safety constraint confirmed by reading `THROW`'s own code directly, not assumed from the glossary alone: with no active `CATCH`, `THROW` jumps straight to `ABORT` - already established in section 3.1 as unsafe to trigger directly, since it resets the return stack and would destroy this whole test framework's own call chain. Every `THROW` in this section's own tests is either wrapped in a genuine, active `CATCH` (via a dedicated internal helper, `TSTTHROWHLP`, defined specifically so it can only ever be invoked that way - never called directly) or is a `THROW(0)` call, confirmed via `THROW`'s own code to be a pure, unconditional no-op that never touches `HANDLER` or `ABORT` at all, making that one specific case safe to call directly without a wrapping `CATCH`.

`TSTCATCHTHROW` specifically has `TSTTHROWHLP` push two dummy values before throwing, then verifies they're genuinely gone afterward - confirming `CATCH`'s own documented "restores data stack depth... on either path" for real, not just that the final result happens to look right. `TSTHANDLERSAVE` verifies the `HANDLER` half of that same documented guarantee directly, reading `HANDLER` itself before and after `CATCH` calls on both the success path (wrapping `DUP`) and the throw path (wrapping `TSTTHROWHLP` again), rather than only inferring it from stack-level behavior the other tests already cover.

Caught and fixed a real depth-check mistake in `TSTHANDLERSAVE` before it was ever inserted - the same class of mistake found via MAME in section 3.13's own `TSTNUMSIGN`: `TSTUAF` is captured immediately after the second `CATCH` call returns, with the thrown code still sitting on the stack alongside `GUARD` (2 cells), not after the subsequent pops that verify it (which would have left only `GUARD`, 1 cell) - the expected depth was originally written as `0`, re-traced carefully and corrected to the true value (`2`) during drafting, the same way the earlier instance of this exact mistake had to be found by the user via a live MAME run instead.

Added a new scratch constant (`TSTHANDSAV`) for saving/restoring the real `HANDLER` across `TSTHANDLERSAVE`'s own dual-path check.

Checked the group wrapper name (`TSTEXCEPTION`) against the whole file before use, per the practice established after the `TSTCOMPARE` collision in an earlier section - safe. Checked every column-1 mnemonic occurrence immediately after insertion,

```
per the practice established after the indentation mistake
found via a failed lwasm run in section 3.13 - zero found. One
real internal-label collision caught by the standard check
(`TZFAIL`/`TZDONE`, colliding with an existing, unrelated
test's own labels from an earlier section), replaced with `TV`.

Verified: `IFEQ`/`ENDC` balance immediately after insertion -
balanced (59/59) and correctly nested end to end on the first
attempt; every one of the 4 tests' own depth-check values
independently re-derived from real arity and confirmed correct
against the final, fully-assembled file state; every real word
reference confirmed to resolve; all 4 tests confirmed wired
into `TSTEXCEPTION`, and `TSTEXCEPTION` itself confirmed wired
into `TSTRUNNER`. Full scenario matrix re-run with
`TSTSELECTOR`'s fifteenth value included (32 scenarios total) -
all pass; explicitly confirmed `TSTSELECTOR=14` includes only
this group's own bodies. Byte-exact split-file reassembly
confirmed. **MAME-CONFIRMED**: the user reports all 4 tests
pass, including `TSTHANDLERSAVE` after its own pre-insertion
depth-check correction - both `CATCH`'s success and exception
paths, `THROW(0)`'s no-op behavior, and `HANDLER`'s correct
restoration across both paths all confirmed correct on real
hardware.
```

- ** `TSTCOMMENTS` added - comments tests (glossary section 3.16, 2 words: `(` and `\`), wired into `TSTRUNNER` and inserted last in the source (after `TSTEXCEPTION`), matching glossary order, gated as `TSTSELECTOR`'s sixteenth value (`-15`). Both words parse from source, so both redirect `SRCADDR` / `SRCLEN` / `TOIN`, matching the established pattern from sections 3.8-3.12.

```
**`(` confirmed to carry real bug-fix history worth verifying
directly, not just noting** - its own code comment documents a
prior version that omitted consuming `WORD`'s own returned
c-addr, leaving a stray value on the data stack after every
`"(...)"` comment, confirmed and fixed via MAME in an earlier
session (symptomatic both as a visible stray value in interpret
mode and as a `-22` CSP mismatch for any colon definition
containing one). This section's own test specifically confirms
the data stack is genuinely unchanged (net 0) after the call,
not just that parsing itself advanced to the right place -
directly re-verifying the fix holds, not merely trusting the
comment.

`\` specifically redirects `SRCLEN` (not just `TOIN`) to a
value distinct from the real terminal input buffer's own
```

length, confirming it genuinely reads `SRCLEN` directly rather than some fixed buffer size - the actual mechanism behind its own documented "follows `SOURCE`... operates correctly inside `EVALUATE`", not just trusting the description.

Caught and fixed a real bug in this section's own first `TSTLPAREN` draft before it was ever inserted: referenced an undefined scratch constant (`TSTSASAV2`) to re-read `SRCADDR` after the call, when `(` in fact never touches `SRCADDR` itself (only `TOIN`) - simplified to check the fake source's own already-known address directly, removing the unnecessary and broken re-read entirely rather than defining a new constant to paper over it.

Checked the group wrapper name (`TSTCOMMENTS`) against the whole file before use, per the practice established after the `TSTCOMPARE` collision in an earlier section - safe. Checked every column-1 mnemonic occurrence immediately after insertion, per the practice established after the indentation mistake found via a failed lwasm run in section 3.13 - zero found. Zero internal-label collisions this time.

Verified: `IFEQ`/`ENDC` balance immediately after insertion - balanced (61/61) and correctly nested end to end on the first attempt; both tests' own depth-check values independently re-derived from real arity and confirmed correct against the final, fully-assembled file state; every real word reference confirmed to resolve, with `LPAREN` itself initially hidden from the automated check by this session's own recurring local-prefix-exclusion artifact, confirmed directly via a literal call-site count rather than trusted blindly; both tests confirmed wired into `TSTCOMMENTS`, and `TSTCOMMENTS` itself confirmed wired into `TSTRUNNER`. Full scenario matrix re-run with `TSTSELECTOR`'s sixteenth value included (34 scenarios total) - all pass; explicitly confirmed `TSTSELECTOR=15` includes only this group's own bodies. Byte-exact split-file reassembly confirmed. Not yet confirmed via MAME.

Structural duplication (identified, some resolved, some not)

- `:` / `CREATE` / `VARIABLE` 's header-building — resolved via `HEADER` (section 7).
- `COMMA` / `CODECOMMA` / `CCOMMA` / `VCOMMA` / `VCCOMMA` / `CCOMMA1` — resolved via `APPENDCELL` / `APPENDBYTE` (section 6).
- `<#` / `#>` recomputing PAD's address inline — resolved, both now call `PADW` (section 20 /

section 6).

- **Identified via the real listing (`forth6809_lst.txt`), not yet acted on: `NUMBERQ` 's inline 32-bit negation (added this session for double-number text input) duplicates `MNEG32` , an existing, already-correct shared routine `DNEGATE` and `DABS` both already call.** Traced precisely why this duplication mattered beyond just code size: `MNEG32` 's instruction order (complement both cells and store both, *then* do the `+1` /carry-propagation step separately afterward) happens to avoid the exact 6809 quirk (`COM` unconditionally sets carry) that `NUMBERQ` 's own, separately hand-written version fell into - which is precisely why `NUMBERQ` 's negation needed two separate bug-fix turns this session while `MNEG32` itself never had the problem at all. Not refactored here - the user's own stated reason for deferring: automated testing should be in place before touching working, duplicated logic, given hand-verifying a refactor across every affected call site the way this session has been doing (MAME tracing turn by turn) doesn't scale well to consolidation work.
- **Identified via the real listing, not yet acted on: the self-referential PFA bug (`CONSTANT` / `DEFER` / `2CONSTANT` / `MARKER`) got four separate, duplicated fixes rather than one shared helper.** Confirmed directly in the listing: `DEFER` 's, `2CONSTANT` 's, and `MARKER` 's fixes are each explicitly commented "same self-referential PFA bug as [`CONSTANT`]," each repeating the identical `LDD CODEHERE` / `ADDD #2` /store sequence independently rather than calling a shared routine. Same deferral reasoning as above - not acted on pending automated testing.
- No further known duplication beyond the two items just above, but the codebase still hasn't been given a full, systematic pass specifically hunting for more - both findings above came from genuinely investigating specific leads, not an exhaustive scan.

Real, load-bearing gaps

- **Software (XON/XOFF) handshaking is still not implemented.** Now that hardware RTS/CTS handshaking is in place, this is the one remaining flow-control gap: this system still never transmits XON/XOFF and would not recognize either byte as a control signal if the remote device sent them — an incoming `$11` / `$13` is just queued as an ordinary character. Only relevant for links with no RTS/CTS wiring (plain 3-wire serial); would need a byte- interception layer between the input ring and `KEY` 's caller.
- **No hard boundary checks** anywhere `DPHERE` / `CODEHERE` / `VARHERE` grow toward each other or toward the stacks. `UNUSED` and `VUNUSED` both correctly *report* the distance to their boundary now (`CODETOP` and the newly-added `APPVARSEND` respectively), but nothing enforces either — a runaway compile can silently corrupt an adjacent region.

- **`VUNUSEDW` (the `VUNUSED` word) computed a meaningless number, not "how much `APPVARS` space remains" - a real, distinct bug, not just the absence of a check. Fixed.** `LDD #APPCODE / SUBD VARHERE` computed `APPCODE - VARHERE` (roughly `$7000` minus wherever `VARHERE` currently sits within `APPVARS`), which had nothing to do with `APPVARS`'s own boundary - looked like a copy-paste slip from `UNUSEDW` immediately above it (`LDD #CODETOP / SUBD CODEHERE`, correct for `CODEHERE`'s own boundary). Surfaced directly by a question about how `APPVARS`'s size is actually set: it wasn't - `APPVARS EQU $021B` defined only its start; there was no end/size constant anywhere. Fixed by adding `APPVARSEND EQU APPVARS+8000` (an exclusive upper bound, `$215B`, matching `CODETOP`'s own convention for `APPCODE`) and changing `VUNUSEDW` to `LDD #APPVARSEND / SUBD VARHERE`. Verified: at cold start (`VARHERE` = `APPVARS`), this computes exactly 8000, matching `APPVARS`'s documented size precisely; zero duplicate symbols; dictionary chain unaffected. `APPVARSEND` initially fell within `APPDICT`'s current range - expected at the time, the same, already-tracked `APPDICT` / `APPVARS` overlap, not something this fix caused. Since resolved by redefining `APPVARSEND` as `APPDICT-1` - see above.

- **`ENVTABLE` is incomplete.** Missing entries: `/HOLD`, `/PAD` (this build never fixed a capacity for either — filling them in would mean deciding a real bound first, not just picking a number), `MAX-D`, `MAX-UD` (need the dispatcher extended to push *two* cells for double-cell answers — current `ENVQUERY` only handles one), `WORDLISTS` / `FLOORED` (need the dispatcher to be able to report a recognized-but-false answer, distinct from "unrecognized name" — no such path exists yet). **RESOLVED - all six entries added and MAME-confirmed this session; see the dedicated entry earlier in this file (search "ENVTABLE completeness"). ENVIRONMENT? is complete.**

- **`CATCH`-wrapped `QUIT` / `INTERPRET` rollback is scoped to one input line only.** A colon definition spanning multiple lines that fails partway through a later line only rolls back that line's contribution, not the whole definition back to `:`. A complete fix needs the region-pointer snapshot taken once at `:` and held until `;` or an error, not refreshed every `QLOOP` pass.

- **RESOLVED - confirmed via the real listing (`forth6809_lst.txt`), not just the source text.** `ABORTHDR`'s `LINK` field is a placeholder `0` in the consolidated/split source — must be set to the real prior `LATEST` value once final ROM layout/assembly order is fixed. `ABORTHDR` doesn't exist as a symbol anymore - renamed to `H_ABORT` and moved into `BASEDICT` along with every other header, as part of the broader dictionary-consolidation work referenced above. `H_ABORT`'s actual `LINK` field is a real, resolved address, not a placeholder: `FDB H_FALSE`, with the source's own comment documenting its history (placeholder `0` -> `H_DUMPW` -> `H_DOESGT` -> `H_DUMPW` again -> `H_FALSE`, tracking the

chain's actual newest entry as the dictionary grew during the session). Independently confirmed by the 224-entry dictionary chain walk from `H_M2` to `0`, checked repeatedly throughout this session - that walk only succeeds if every `LINK` field in the chain, including this one, is a real, correct address; a placeholder `0` mid-chain would have broken it outright. No action needed on the source.

Never resolved after being explicitly raised

- **`J` doesn't generalize past one level of loop nesting.** No `J2`/deeper equivalent exists. A triple-nested loop's innermost body has no built-in way to reach the outermost index without manually replicating `J`'s offset arithmetic one frame further.
- **`LEAVE`'s correctness depends on always being textually inside the loop it affects** — never verified against a `LEAVE` called from a separately-defined word invoked from within a loop (which would read/set the wrong stack frame, or crash).
- **RESOLVED via direct code inspection of the real listing (`forth6809_lst.txt`), not just address arithmetic - the underlying inconsistency is now fully understood, though the answer is a genuine mix, not a simple "fixed" or "still open."** Traced `CHARW` (`CHAR`) and `BRACKCHAR` (`[CHAR]`) precisely: both simply push a space delimiter, `JSR WORD`, and read the first character of whatever comes back - no separate cap logic of their own at all. They automatically, correctly inherited `WORD`'s new 46-character cap (`WORDMAXCHARS`) with zero code changes needed - this part of the original item is stale. `FIND` also has no independent cap of its own - traced its comparison down to `HDRFLAGS ANDA #$1F`, masking the dictionary header's `LEN/FL` byte to its low 5 bits. This is the real finding: dictionary header names genuinely are still capped at 31 characters, but not because they "inherit `WORD`'s cap" as the original item claimed - the header format itself allocates only 5 bits (bits 4-0) for the name length, an absolute maximum of 31 regardless of anything `WORD` does or how it's redesigned. That's a separate, permanent architectural constraint (the header storage format), not a leftover inconsistency from `WORD`'s own redesign. The original item's framing was imprecise even when written - it grouped "things that call `WORD`" (`CHAR`/`[CHAR]`, no independent cap) together with "things bounded by the header format" (header names/`FIND`) as if they were the same constraint, when they're two different ones. Original text: **`WORD`'s 31-character cap is now inconsistent within the system.** `PARSE`/`PARSE-NAME` were deliberately rewritten to not share it, but `WORD` itself — and everything still built on it (`CHAR`, `[CHAR]`, header names, `FIND`) — still has it.
- **The optional Search-Order word set is not implemented.** Every word resolves through one unified dictionary chain rooted at `LATEST`; there is no multi-wordlist support (`WORDLIST`, `GET-ORDER`/`SET-ORDER`, `ALSO`/`ONLY`, etc.). This was a foundational

decision fixed since `COLDSTRT`'s first version, not an oversight — now documented explicitly (ClaudeForth documentation, Section 4.5) along with what adding it later would actually require: restructuring `FIND` into a loop over an active search order, changing `HEADER` to link into a selectable "current" wordlist instead of unconditionally into `LATEST`, and fixing the few words (`RECURSE`, `;`'s unsmudge step, `WORDS`) that read `LATEST` directly today.

Open design questions (not defects — deliberate unresolved choices)

- Whether `ALLOT` / `VALLOT` / `PICK` / `ROLL` / `BASE` should range-check their inputs against actual stack/region bounds. Currently none of them do, consistently, by choice rather than oversight.
- Whether any additional distinction like `TIB` / `SOURCE` (fixed terminal buffer vs. current input source) is needed anywhere else in the system where `EVALUATE`'s redirection might still cause a mismatch.
- `SWIH`'s throw code (`-99` in the consolidated source) was never assigned a real, deliberate value — it's a placeholder for "some hardware trap occurred," not a considered ANS-style code.
- `REPLACES` / `SUBSTITUTE` remain single-slot (one registered name/value pair, overwritten by the next `REPLACES` call) rather than a true multi-entry table. A real table needs its own storage layout and lookup structure — a deliberate scope decision made when `SUBSTITUTE` was completed, not an oversight.

What's solid (for reference, not action)

Stack manipulation (Core + Core Ext + return-stack transfer), all arithmetic (single + double + mixed precision), logic, comparison (single + double), full control flow including `CASE`, all defining words including `DEFER` / `MARKER` / `VALUE` / `TO` (`VALUE` now correctly targeting mutable space), memory and string operations (including a completed `SUBSTITUTE`), `SOURCE` / `EVALUATE` / `REFILL` mechanics, and the Tools word set (`.S` / `WORDS` / `DUMP`, with `DUMP`'s edge-case bug fixed) are complete and were traced/verified against concrete cases during the build, not merely asserted correct.